AImsII

Irenaeus Wolff

2026-02-06

```r
library(dplyr)
library(Exact)
library(fixest)
library(ggplot2)
library(tidyr)
library(texreg)
library(scales)
```

Load data

```r
# load cleaned dataset
choices <- read.delim(
↪  "~/work/zz_nextcloud/y_AImsAndAixperience/Experiments/ProlificData/files_Fri_02-06-2026_15-28-25-48
↪  )
choicesP1 <- subset(choices, Part == 1)
subjects <- read.csv(
↪  "~/work/zz_nextcloud/y_AImsAndAixperience/Experiments/ProlificData/files_Fri_02-06-2026_15-28-25-48
↪  , header =
↪  T)
prolificData <- read.csv(
↪  "~/work/zz_nextcloud/y_AImsAndAixperience/Experiments/ProlificData/files_Fri_02-06-2026_15-28-25-48
↪  )
```

Exclude observations based on preregistration

```r
prolificData$logCompleTime <- log(prolificData$Time.taken)
prolificData$tooFast <- 1*(prolificData$logCompleTime <
↪  mean(log(prolificData$Time.taken)) - 2*sd(log(prolificData$Time.taken)))

choices <- subset(choices, BotCheck1attempt == 1 & Part1CQsPassedOnAttempt <= 2 &
↪  Part2CQsPassedOnAttempt <= 2 & Reliable >= 6 & AIUsageInStudy == 0 & Delegated >= 0
↪  & ProlificID %in% prolificData$Participant.id[prolificData$tooFast == 0])
subjects <- subset(subjects, ProlificID %in% choices$ProlificID)
choicesP1 <- subset(choicesP1, ProlificID %in% choices$ProlificID)
table(subjects$EVAlgorithm)
```

```
##
##   0   1
## 115 118
```

```r
# STILL TO BE DONE: Exclusion by log time (-> TotalSecondsInExperiment)
```

overview delegation and (non-)dominated choices

```r
choices <- choices %>%
  mutate(Algorithm = ifelse(EVAlgorithm == 1, "EV", "Pref"))

choicesAv <- choices %>%
  filter(Part == 2) %>%
    group_by(Algorithm, Part, WithinPartDecisionNumber) %>%
    summarise(
      avDeleg = mean(Delegated, na.rm = T),
      avNonDom = mean( Nondominated[Delegated == 0 & Option > 0], na.rm = T)
  )
```

```
## `summarise()` has grouped output by 'Algorithm', 'Part'. You can override using
## the `.groups` argument.
```

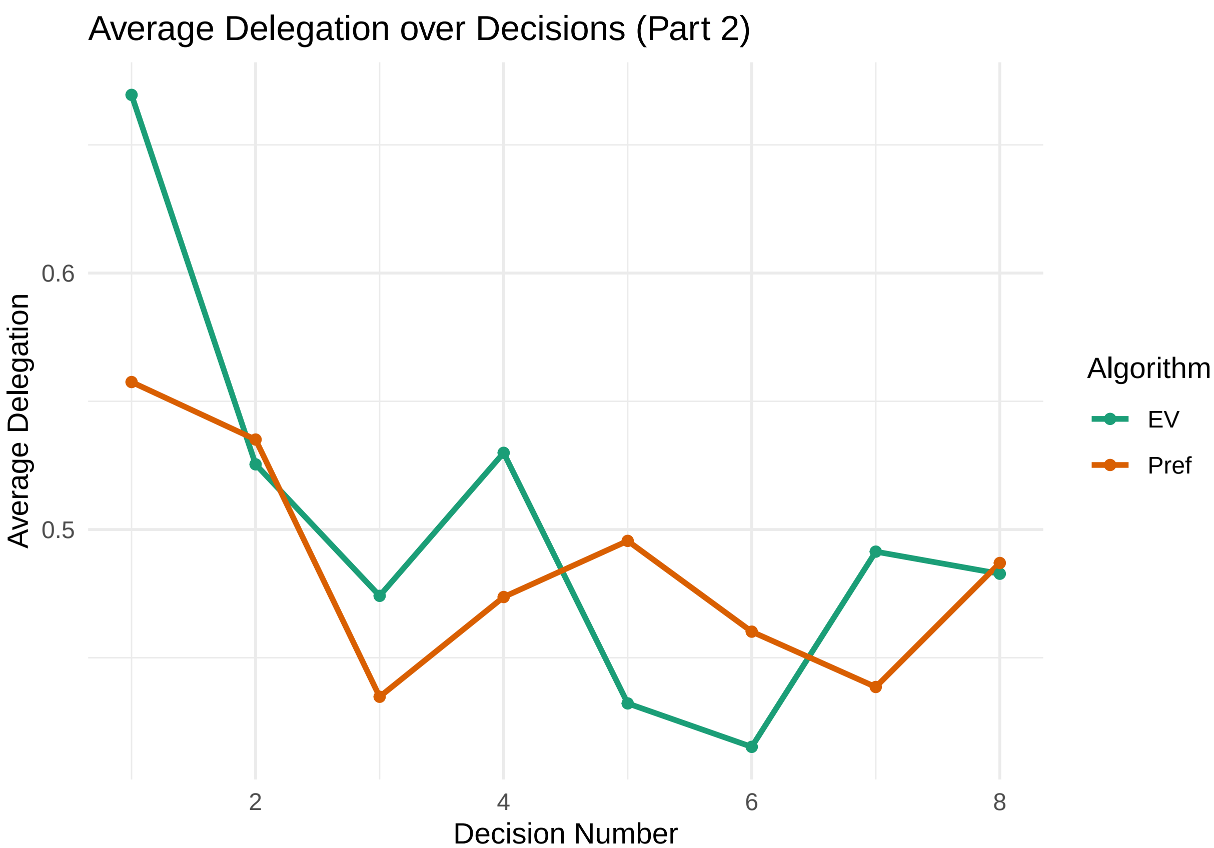
```r
choicesAv
```

```
## # A tibble: 16 x 5
## # Groups:   Algorithm, Part [2]
##    Algorithm  Part WithinPartDecisionNumber avDeleg avNonDom
##    <chr>     <int>                    <int>   <dbl>    <dbl>
##  1 EV            2                        1   0.669    0.342
##  2 EV            2                        2   0.525    0.214
##  3 EV            2                        3   0.474    0.3
##  4 EV            2                        4   0.530    0.333
##  5 EV            2                        5   0.432    0.269
##  6 EV            2                        6   0.415    0.464
##  7 EV            2                        7   0.491    0.356
##  8 EV            2                        8   0.483    0.433
##  9 Pref          2                        1   0.558    0.4
## 10 Pref          2                        2   0.535    0.327
## 11 Pref          2                        3   0.435    0.344
## 12 Pref          2                        4   0.474    0.267
## 13 Pref          2                        5   0.496    0.429
## 14 Pref          2                        6   0.460    0.333
## 15 Pref          2                        7   0.439    0.391
## 16 Pref          2                        8   0.487    0.322
```

```r
choices %>%
  filter(Part == 2) %>%
    group_by(Algorithm) %>%
    summarise(
      avNonDom = mean( Nondominated[Delegated == 0 & Option > 0], na.rm = T)
  )
```

```
## # A tibble: 2 x 2
##   Algorithm avNonDom
##   <chr>        <dbl>
## 1 EV           0.341
## 2 Pref         0.351
```

```r
choicesAv %>%
  ggplot(aes(
    x = WithinPartDecisionNumber,
    y = avDeleg,
    color = factor(Algorithm),
    group = Algorithm
  )) +
  geom_line(linewidth = 1) +
  geom_point() +
  labs(
    x = "Decision Number",
    y = "Average Delegation",
    color = "Algorithm",
    title = "Average Delegation over Decisions (Part 2)"
  ) + scale_color_brewer(palette = "Dark2") +
#   scale_color_manual(
#     values = c("#4E79A7", "#59A14F")
# ) +
  theme_minimal()
```



NB: at least 65% suboptimal choices without delegation

Hypothesis testing

```r
tab <- table(
  Delegated = choices$Delegated[choices$Part == 2 & choices$WithinPartDecisionNumber ==
  ↪  1],
```

```r
  EVAlgorithm = choices$EVAlgorithm[choices$Part == 2 & choices$WithinPartDecisionNumber
  ↪  == 1]
)

prop.table(tab, margin = 2)
```

```
##          EVAlgorithm
## Delegated         0         1
##         0 0.4424779 0.3305085
##         1 0.5575221 0.6694915
```

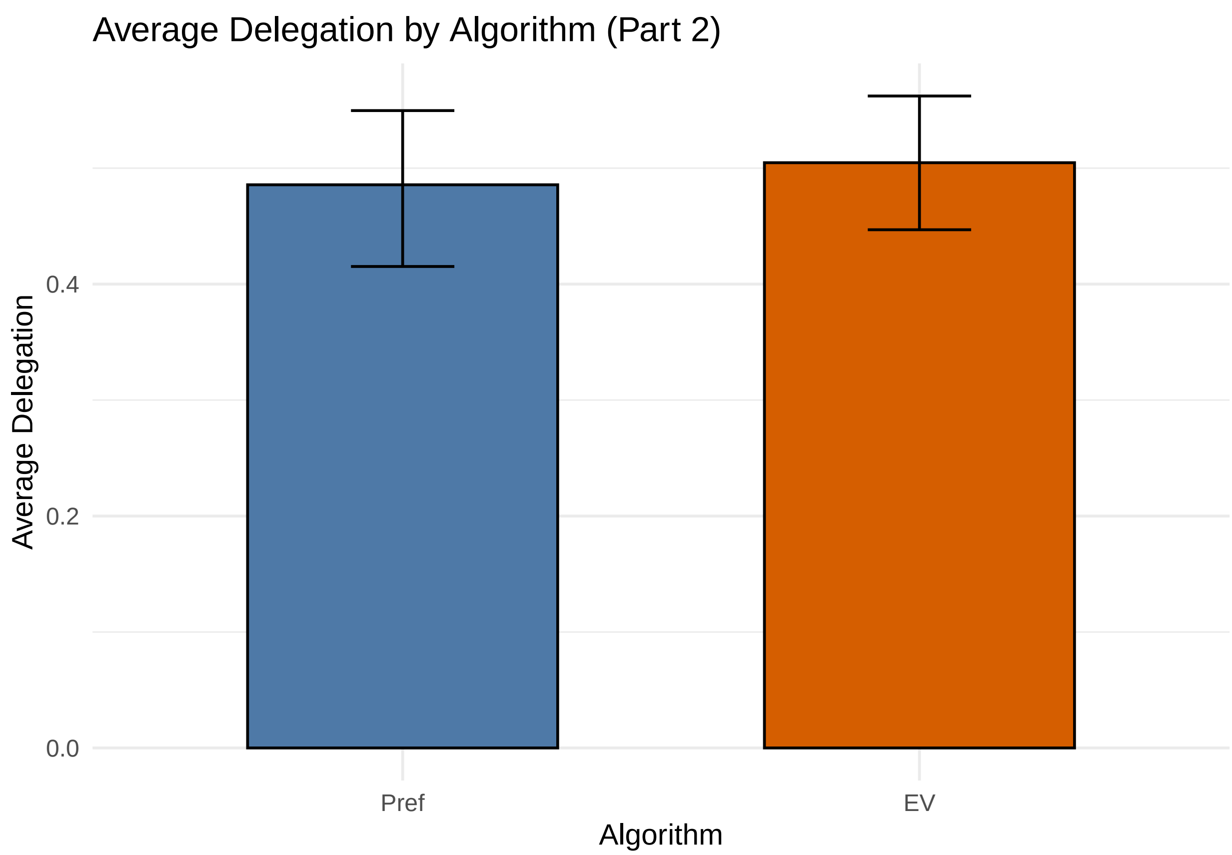
```r
exact.test(tab, method = "Boschloo", alternative = "greater", to.plot = F)$p.value
```

```
## [1] 0.0412565
```

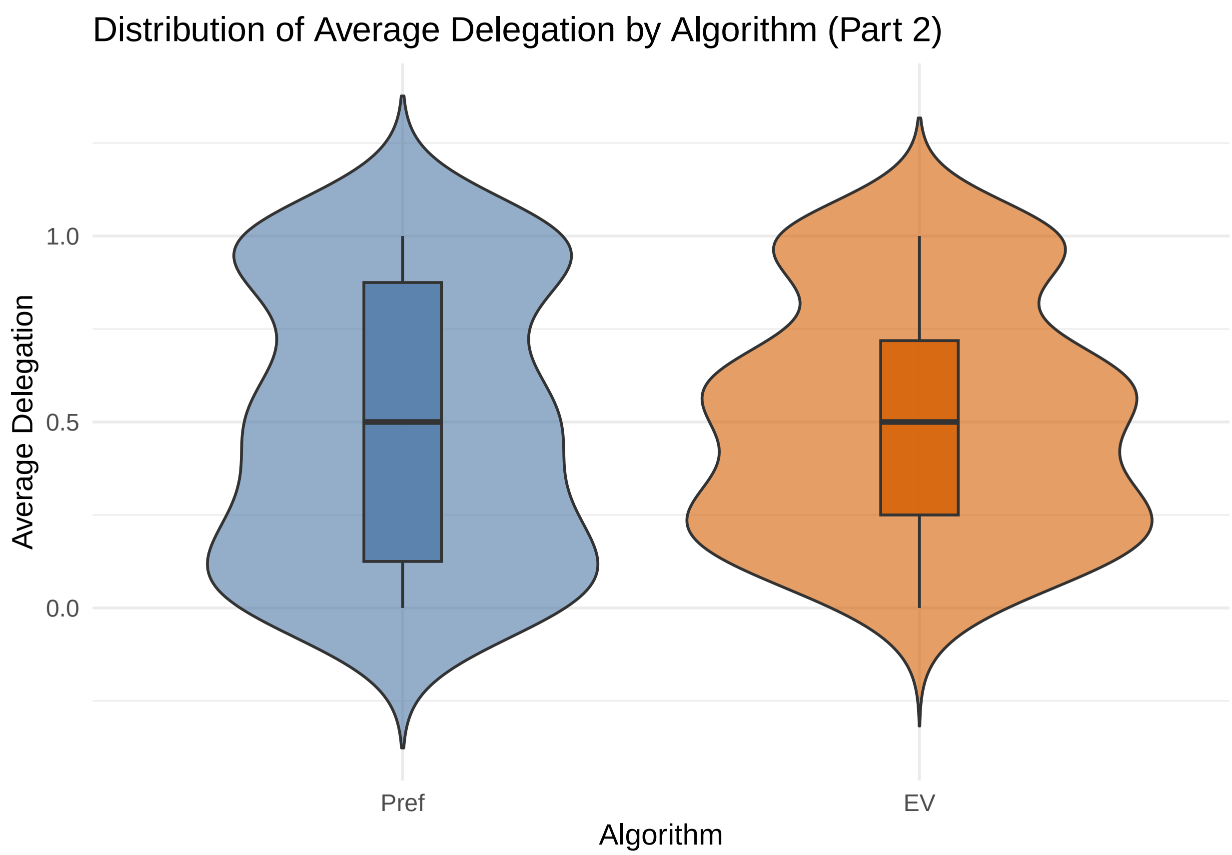
```r
delegAv <- choices %>%
  filter(Part == 2) %>%
    group_by(Algorithm, ProlificID) %>%
    summarise(
      avDeleg = mean(Delegated, na.rm = T),
  ) %>% mutate(Algorithm = factor(Algorithm, levels = c("Pref", "EV")))
```

```
## `summarise()` has grouped output by 'Algorithm'. You can override using the
## `.groups` argument.
```

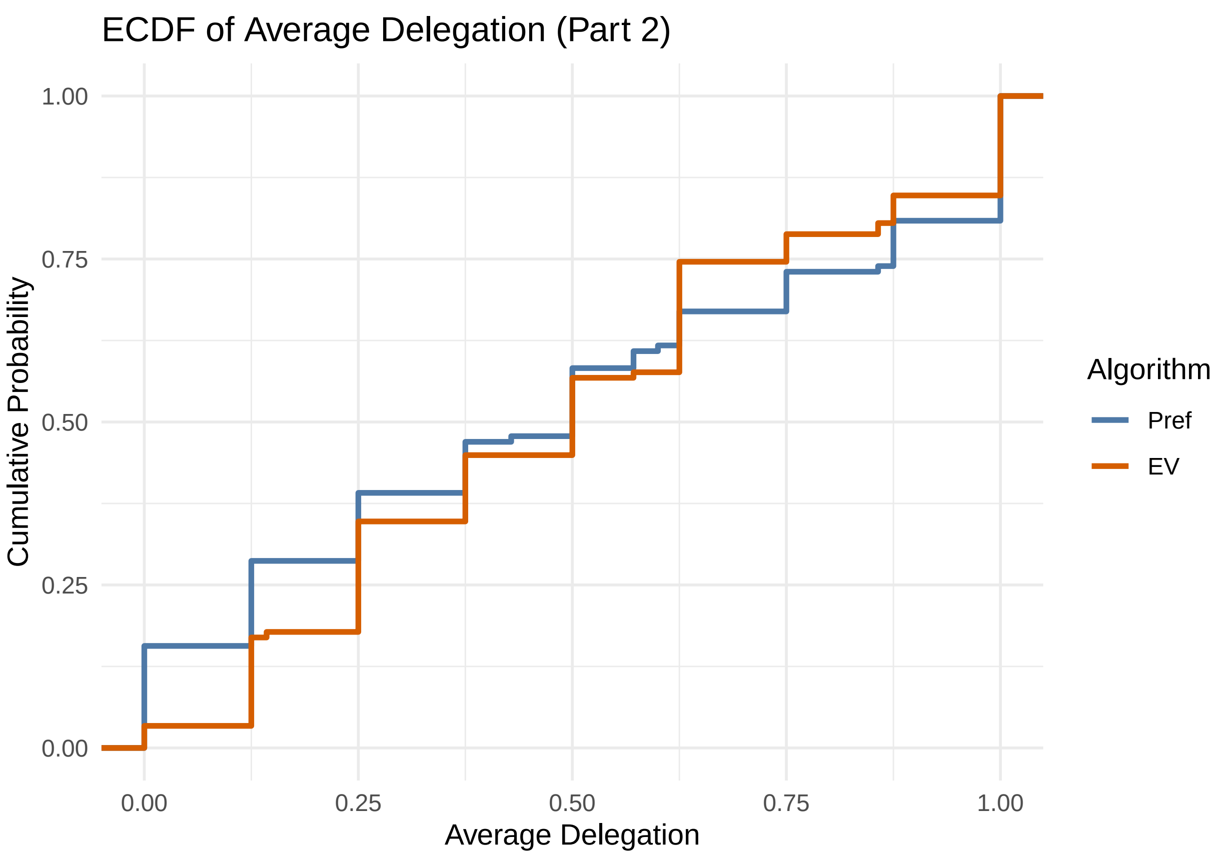
```r
delegAv %>%
  ggplot(aes(x = Algorithm, y = avDeleg, fill = Algorithm)) +
  stat_summary(
    fun = mean,
    geom = "bar",
    width = 0.6,
    color = "black"
  ) +
  stat_summary(
    fun.data = mean_cl_boot,
    geom = "errorbar",
    width = 0.2
  ) +
  scale_fill_manual(
    values = c("Pref" = "#4E79A7", "EV" = "#D55E00")
  ) +
  labs(
    x = "Algorithm",
    y = "Average Delegation",
    title = "Average Delegation by Algorithm (Part 2)"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```




```r
delegAv %>%
  ggplot(aes(x = Algorithm, y = avDeleg, fill = Algorithm)) +
  geom_violin(trim = FALSE, alpha = 0.6) +
  geom_boxplot(
    width = 0.15,
    outlier.shape = NA,
    alpha = 0.8
  ) +
  scale_fill_manual(
    values = c("Pref" = "#4E79A7", "EV" = "#D55E00")
  ) +
  labs(
    x = "Algorithm",
    y = "Average Delegation",
    title = "Distribution of Average Delegation by Algorithm (Part 2)"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```




```r
delegAv %>%
  ggplot(aes(x = avDeleg, color = Algorithm)) +
  stat_ecdf(linewidth = 1) +
  scale_color_manual(
    values = c("Pref" = "#4E79A7", "EV" = "#D55E00")
  ) +
  labs(
    x = "Average Delegation",
    y = "Cumulative Probability",
    color = "Algorithm",
    title = "ECDF of Average Delegation (Part 2)"
  ) +
  theme_minimal()
```




```r
wilcox.test(
  avDeleg ~ Algorithm,
  data = delegAv,
  alternative = "less"
)
```

```
##
##  Wilcoxon rank sum test with continuity correction
##
## data:  avDeleg by Algorithm
## W = 6428, p-value = 0.2425
## alternative hypothesis: true location shift is less than 0
```

```r
# LPM with FE and cluster
delReg <- feols(
  Delegated ~ EVAlgorithm*WithinPartDecisionNumber | OptionsetID,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2)
)
summary(delReg)
```

```
## OLS estimation, Dep. Var.: Delegated
## Observations: 1,848
## Fixed-effects: OptionsetID: 8
## Standard-errors: Clustered (ProlificID)
##                                         Estimate Std. Error  t value Pr(>|t|)
## EVAlgorithm                             0.072905   0.053131  1.37216 0.171338
```

```
## WithinPartDecisionNumber             -0.009663   0.005675 -1.70260 0.089982 .
## EVAlgorithm:WithinPartDecisionNumber -0.012349   0.008293 -1.48906 0.137830
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.495714     Adj. R2: 0.011578
##                  Within R2: 0.006493
```

```
delRegPlainTrmt <- feols(
  Delegated ~ EVAlgorithm | OptionsetID,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2)
)
summary(delRegPlainTrmt)
```

```
## OLS estimation, Dep. Var.: Delegated
## Observations: 1,848
## Fixed-effects: OptionsetID: 8
## Standard-errors: Clustered (ProlificID)
##             Estimate Std. Error  t value Pr(>|t|)
## EVAlgorithm 0.017567   0.043886 0.400297  0.68931
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.497254     Adj. R2: 0.006511
##                  Within R2: 3.119e-4
```

```
delRegPlainTrmtNoP1 <- feols(
  Delegated ~ EVAlgorithm | OptionsetID,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2 & WithinPartDecisionNumber > 1)
)
summary(delRegPlainTrmtNoP1)
```

```
## OLS estimation, Dep. Var.: Delegated
## Observations: 1,617
## Fixed-effects: OptionsetID: 8
## Standard-errors: Clustered (ProlificID)
##             Estimate Std. Error  t value Pr(>|t|)
## EVAlgorithm 0.002496   0.044616 0.055946  0.95543
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.495957     Adj. R2: 0.00908
##                  Within R2: 6.329e-6
```

```
delRegP1 <- feols(
  Delegated ~ EVAlgorithm | OptionsetID,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2 & WithinPartDecisionNumber == 1)
)
summary(delRegP1)
```

```
## OLS estimation, Dep. Var.: Delegated
## Observations: 231
## Fixed-effects: OptionsetID: 8
## Standard-errors: Clustered (ProlificID)
##             Estimate Std. Error t value Pr(>|t|)
```

```
## EVAlgorithm 0.112868     0.0648 1.74178 0.082884 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.474644     Adj. R2: 0.0145
##                  Within R2: 0.013716
```

```
choices <- choices %>%
  arrange(ProlificID, WithinPartDecisionNumber) %>%
  group_by(ProlificID) %>%
  mutate(
    # Indikatoren
    delegated_round = (Delegated == 1),
    zero_payoff = (RelevantPayoff == 0 & Delegated == 1),

    # Kumulative Summen (nur Vergangenheit!)
    cum_delegated = lag(cumsum(delegated_round), default = 0),
    cum_zero = lag(cumsum(zero_payoff), default = 0),

    # Rate (optional)
    zero_rate = ifelse(cum_delegated > 0,
                       cum_zero / cum_delegated,
                       NA)
  ) %>%
  ungroup()

delRegPreReg <- feols(
  Delegated ~ EVAlgorithm*(WithinPartDecisionNumber + zero_rate) | OptionsetID,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2)
)
```

```
## NOTE: 524 observations removed because of NA values (RHS: 524).
```

```
summary(delRegPreReg)
```

```
## OLS estimation, Dep. Var.: Delegated
## Observations: 1,324
## Fixed-effects: OptionsetID: 8
## Standard-errors: Clustered (ProlificID)
##                                       Estimate Std. Error   t value   Pr(>|t|)
## EVAlgorithm                          -0.048717   0.083122 -0.586090 5.5845e-01
## WithinPartDecisionNumber             -0.001795   0.008257 -0.217380 8.2813e-01
## zero_rate                            -0.385350   0.078139 -4.931567 1.6664e-06
## EVAlgorithm:WithinPartDecisionNumber  0.001877   0.011504  0.163119 8.7058e-01
## EVAlgorithm:zero_rate                -0.049677   0.097673 -0.508609 6.1157e-01
##
## EVAlgorithm
## WithinPartDecisionNumber
## zero_rate                            ***
## EVAlgorithm:WithinPartDecisionNumber
## EVAlgorithm:zero_rate
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.467127     Adj. R2: 0.115951
##                  Within R2: 0.107265
```

```r
choicesP1 <- choicesP1 %>%
  mutate(EVChoice = case_when(
    grepl("^P", ChoiceType) & PBetChosen == 1 ~ 1,
    grepl("^D", ChoiceType) & PBetChosen == 0 ~ 1,
    TRUE ~ 0
  ))

choicesP1 <- choicesP1 %>%
  mutate(truePChoice = case_when(
    grepl("^D", ChoiceType) & PBetChosen == 1 ~ 1,
    TRUE ~ 0
  ))

choices <- choices %>%
  left_join(
    choicesP1 %>%
      group_by(ProlificID) %>%
      summarise(EVChoices = sum(EVChoice), .groups = "drop"),
    by = c("ProlificID")
  )

choices <- choices %>%
  left_join(
    choicesP1 %>%
      group_by(ProlificID) %>%
      summarise(truePChoices = sum(truePChoice), .groups = "drop"),
    by = c("ProlificID")
  )

choices <- choices %>%
  left_join(
    choicesP1 %>% select(ProlificID, OptionsetID, Confidence),
    by = c("ProlificID", "OptionsetID")
  )

# Schritt 1: Anzahl der P105-Einträge pro ProlificID in choicesP1 berechnen
PBetCounts <- choicesP1 %>%
  group_by(ProlificID) %>%
  summarise(PBet_count = sum(PBetChosen))

# Schritt 2: Diese Anzahl in choices eintragen (per left_join)
choices <- choices %>%
  left_join(PBetCounts, by = "ProlificID")

choices <- choices %>%
  left_join(
    subjects %>%
      select(ProlificID, Female, Age, Education, HHIncome, ProbOfBadComputerChoices,
       ↪  ProbOfAbsurdComputerChoices, ComputerRiskTaking),
    by = c("ProlificID" = "ProlificID")
  )

choices <- choices %>%
```

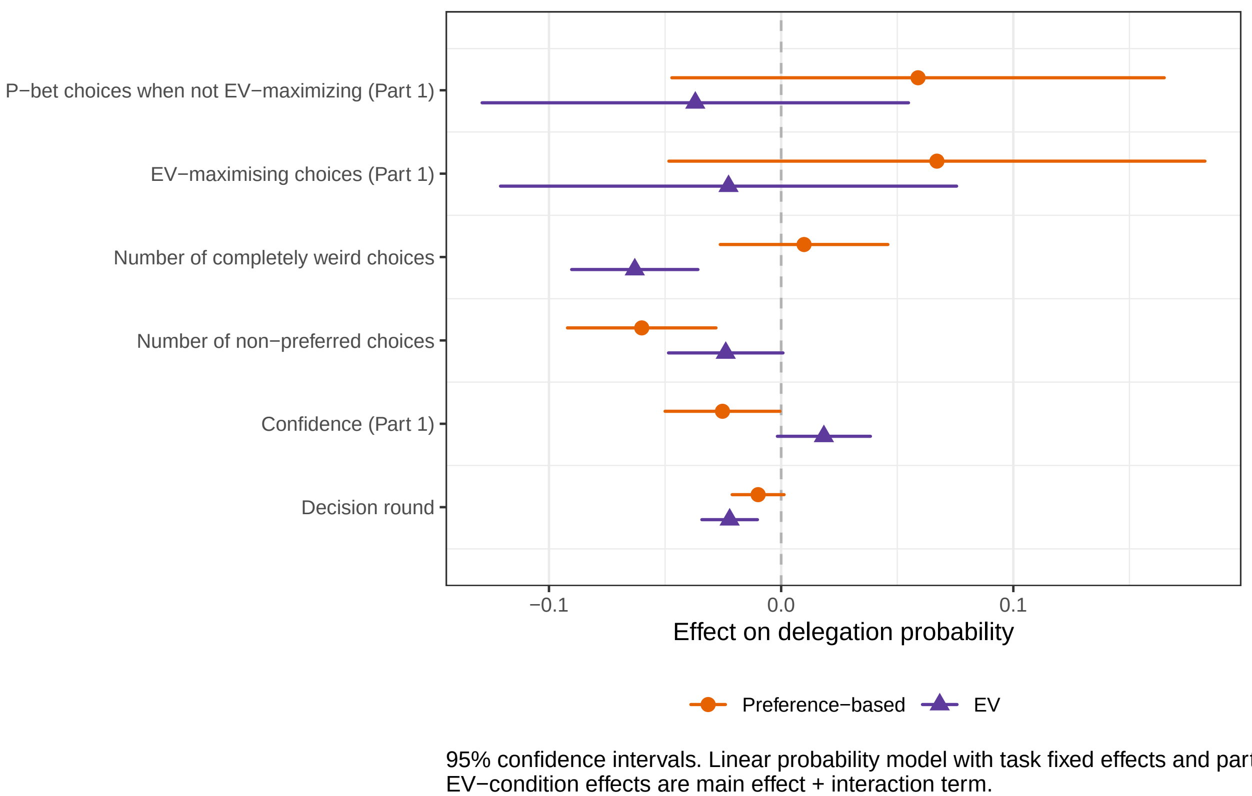
```r
  left_join(
    choicesP1 %>% select(ProlificID, OptionsetID, ConfidenceP1 = Confidence),
    by = c("ProlificID", "OptionsetID")
  )

delRegWConf <- feols(
  Delegated ~ EVAlgorithm*(WithinPartDecisionNumber+ConfidenceP1 +
↪  ProbOfAbsurdComputerChoices + ProbOfBadComputerChoices + EVChoices + truePChoices) |
↪  OptionsetID,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2)
)
summary(delRegWConf)
```

```
## OLS estimation, Dep. Var.: Delegated
## Observations: 1,848
## Fixed-effects: OptionsetID: 8
## Standard-errors: Clustered (ProlificID)
##                                          Estimate Std. Error   t value
## EVAlgorithm                              0.856469   0.891957  0.960213
## WithinPartDecisionNumber                -0.009920   0.005711 -1.737089
## ConfidenceP1                            -0.025275   0.012620 -2.002676
## ProbOfAbsurdComputerChoices              0.009864   0.018425  0.535348
## ProbOfBadComputerChoices                -0.060048   0.016324 -3.678547
## EVChoices                                0.067083   0.058904  1.138847
## truePChoices                             0.058960   0.054107  1.089695
## EVAlgorithm:WithinPartDecisionNumber    -0.012262   0.008341 -1.470058
## EVAlgorithm:ConfidenceP1                 0.043682   0.015740  2.775229
## EVAlgorithm:ProbOfAbsurdComputerChoices -0.072908   0.023059 -3.161780
## EVAlgorithm:ProbOfBadComputerChoices     0.036186   0.020603  1.756318
## EVAlgorithm:EVChoices                   -0.089742   0.077330 -1.160496
## EVAlgorithm:truePChoices                -0.095940   0.071561 -1.340675
##                                          Pr(>|t|)
## EVAlgorithm                             0.3379475
## WithinPartDecisionNumber                0.0836987 .
## ConfidenceP1                            0.0463759 *
## ProbOfAbsurdComputerChoices             0.5929221
## ProbOfBadComputerChoices                0.0002916 ***
## EVChoices                               0.2559415
## truePChoices                            0.2769783
## EVAlgorithm:WithinPartDecisionNumber    0.1429009
## EVAlgorithm:ConfidenceP1                0.0059660 **
## EVAlgorithm:ProbOfAbsurdComputerChoices 0.0017772 **
## EVAlgorithm:ProbOfBadComputerChoices    0.0803535 .
## EVAlgorithm:EVChoices                   0.2470398
## EVAlgorithm:truePChoices                0.1813374
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.474486     Adj. R2: 0.089466
##                  Within R2: 0.089764
```

```r
# All regression coefficients with condition-specific marginal effects
coefs_wc <- coef(delRegWConf)
vcm_wc   <- vcov(delRegWConf)
```

```r
# Helper function for computing SE of sum of two coefficients
se_sum <- function(v1, v2) sqrt(vcm_wc[v1, v1] + vcm_wc[v2, v2] + 2 * vcm_wc[v1, v2])

# Map coefficient names with readable labels
coef_spec <- data.frame(
  var_name = c("WithinPartDecisionNumber", "ConfidenceP1", "ProbOfBadComputerChoices",
               "ProbOfAbsurdComputerChoices", "EVChoices", "truePChoices"),
  var_label = c("Decision round", "Confidence (Part 1)",
                "Number of non-preferred choices", "Number of completely weird choices",
                "EV-maximising choices (Part 1)", "P-bet choices when not EV-maximizing
                 ↪  (Part 1)"),
  stringsAsFactors = FALSE
)

# Build results data frame
plot_data <- data.frame()
for (i in seq_len(nrow(coef_spec))) {
  var_name <- coef_spec$var_name[i]
  var_label <- coef_spec$var_label[i]

  # Preference-based condition (main effect)
  pb_est <- coefs_wc[var_name]
  pb_se  <- sqrt(vcm_wc[var_name, var_name])

  # EV condition (main + interaction)
  int_name <- paste0("EVAlgorithm:", var_name)
  if (int_name %in% names(coefs_wc)) {
    ev_est <- pb_est + coefs_wc[int_name]
    ev_se  <- se_sum(var_name, int_name)
  } else {
    ev_est <- pb_est
    ev_se  <- pb_se
  }

  plot_data <- rbind(plot_data, data.frame(
    Variable = var_label,
    Condition = c("Preference-based", "EV"),
    Estimate = c(pb_est, ev_est),
    SE = c(pb_se, ev_se),
    stringsAsFactors = FALSE
  ))
}

plot_data <- plot_data %>%
  mutate(
    Variable = factor(Variable, levels = coef_spec$var_label),
    Condition = factor(Condition, levels = c("Preference-based", "EV")),
    CI_lo = Estimate - 1.96 * SE,
    CI_hi = Estimate + 1.96 * SE,
    y_pos = as.numeric(Variable) + ifelse(Condition == "Preference-based", 0.15, -0.15)
  )
```

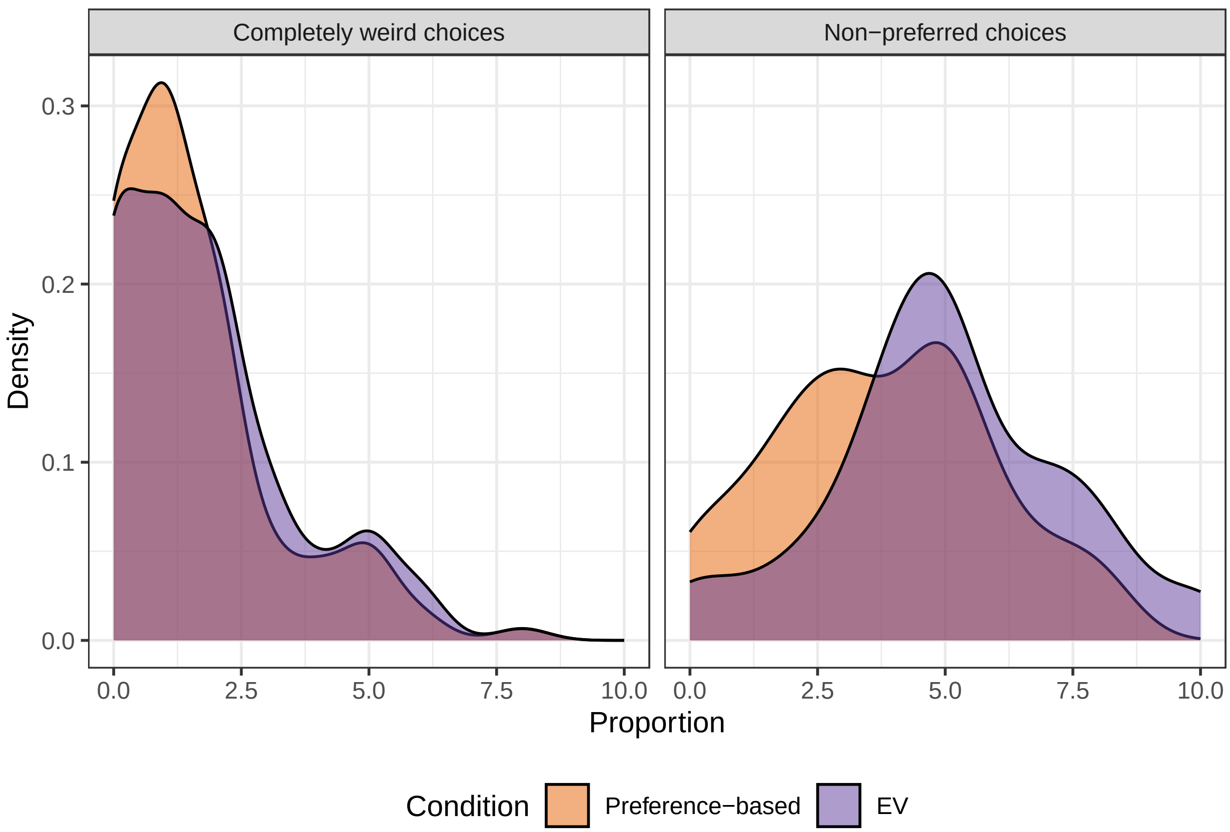
```r
p_all <- ggplot(plot_data) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "gray70") +
  geom_linerange(aes(xmin = CI_lo, xmax = CI_hi, y = y_pos, color = Condition),
                 linewidth = 0.6, lineend = "round") +
  geom_point(aes(x = Estimate, y = y_pos, color = Condition, shape = Condition),
             size = 2.5) +
  scale_y_continuous(breaks = seq_len(nrow(coef_spec)),
                     labels = levels(plot_data$Variable),
                     expand = c(0.15, 0)) +
  scale_color_manual(values = c("Preference-based" = "#E66100", "EV" = "#5D3A9B")) +
  scale_shape_manual(values  = c("Preference-based" = 16, "EV" = 17)) +
  labs(x = "Effect on delegation probability",
       y = NULL, color = NULL, shape = NULL,
       caption = "95% confidence intervals. Linear probability model with task fixed
       ↪  effects and participant-clustered SEs.\nEV-condition effects are main effect +
       ↪  interaction term.") +
  theme_bw(base_size = 10) +
  theme(legend.position = "bottom",
        panel.grid.major.y = element_blank(),
        plot.caption = element_text(hjust = 0, size = 9))

p_all
```




```r
ggsave("CoefPlotBadAbsurd.pdf", p_all, width = 7, height = 4.5)
```

The different drivers are interesting, as the following plots show that people expect (slightly) /more/ weird and bad choices under /EV/

```r
p_prob <- subjects %>%
  pivot_longer(cols = c(ProbOfBadComputerChoices, ProbOfAbsurdComputerChoices),
               names_to = "variable", values_to = "value") %>%
  ggplot(aes(x = value, fill = factor(EVAlgorithm))) +
  geom_density(alpha = 0.5) +
  facet_wrap(~ variable, labeller = labeller(variable = c(
    ProbOfBadComputerChoices    = "Non-preferred choices",
    ProbOfAbsurdComputerChoices = "Completely weird choices"
  ))) +
  scale_fill_manual(values = c("0" = "#E66100", "1" = "#5D3A9B"),
                    labels = c("0" = "Preference-based", "1" = "EV"),
                    name = "Condition") +
  labs(x = "Proportion", y = "Density") +
  theme_bw() +
  theme(legend.position = "bottom")

p_prob
```




```r
ggsave("ExpectedBadAndAbsurdChoicesByTrmt.pdf", p_prob, width = 7, height = 4)
```

the non-delegators

```r
choices <- choices %>%
  group_by(ProlificID) %>%
  mutate(
    sumDelegations = sum(Delegated)
```

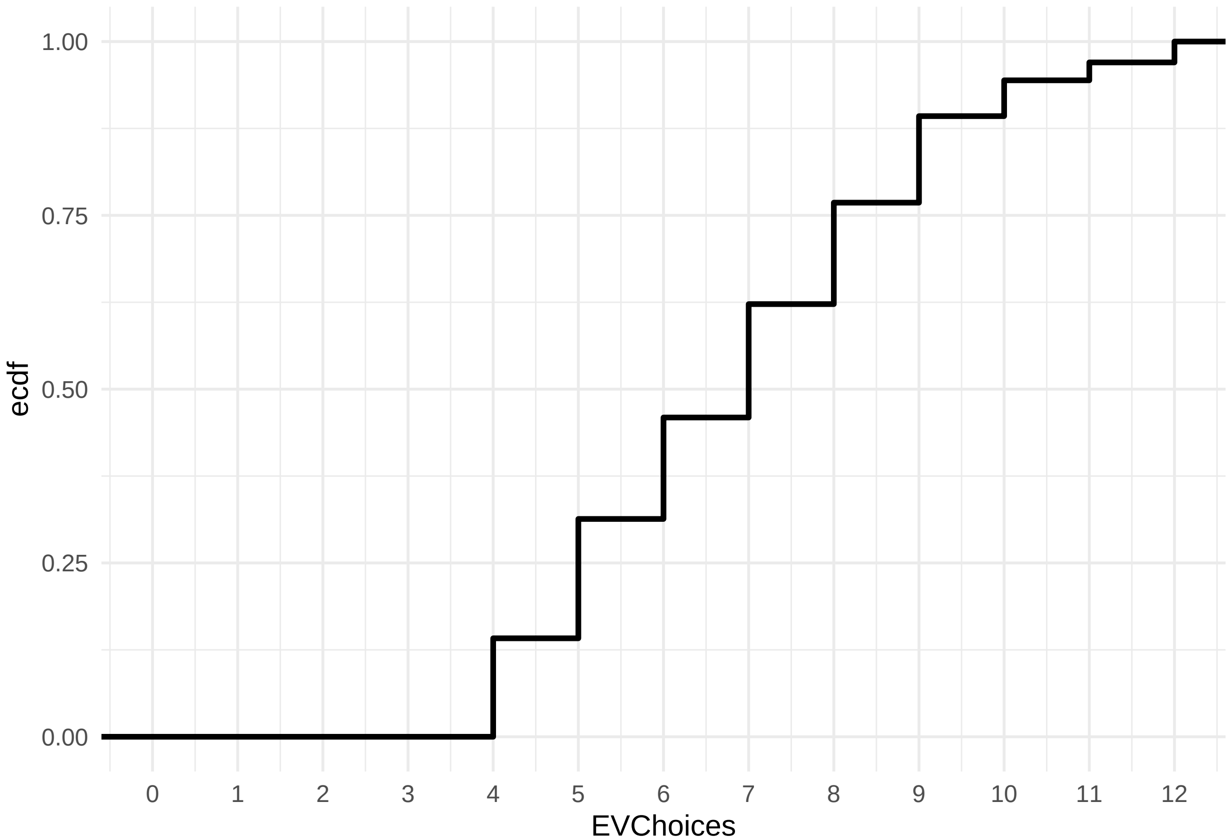
```r
  ) %>%
  ungroup()

choices <- choices %>%
  left_join(
    prolificData %>%
      select(Participant.id, TimeInSeconds = Time.taken),
    by = c("ProlificID" = "Participant.id")
  )

TimeAndDelegations <- choices %>%
  select(ProlificID, TimeInSeconds, sumDelegations) %>%
  distinct()

cor.test(TimeAndDelegations$TimeInSeconds, TimeAndDelegations$sumDelegations)
```

```
##
##  Pearson's product-moment correlation
##
## data:  TimeAndDelegations$TimeInSeconds and TimeAndDelegations$sumDelegations
## t = 0.099062, df = 231, p-value = 0.9212
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
##  -0.1221061  0.1349261
## sample estimates:
##         cor
## 0.006517667
```

not more conscientious judging by time taken.

the EV-maxxers

```r
# EVChoices introduced in chunk Hypothesis testing
choices %>%
  distinct(ProlificID, .keep_all = TRUE) %>%
  ggplot(aes(x = EVChoices)) +
  stat_ecdf(linewidth = 1) +
  scale_x_continuous(breaks = 0:12, limits = c(0, 12)) +
  theme_minimal()
```



There is heterogeneity. About 25% of people choose the EV-maximizing option at least 9/12 times, about 37.5% at least 8/12 times…

```
# 1. Daten vorbereiten: Pro Teilnehmer nur eine Zeile mit sumDelegations und den
↪  gewünschten Variablen
analysis_data <- choices %>%
  group_by(ProlificID) %>%
  summarise(
    EVAlgorithm = first(EVAlgorithm),
    sumDelegations = first(sumDelegations),
    Female = first(Female),
    Age = first(Age),
    HHIncome = first(HHIncome),
    Education = first(Education),
    ProbOfBadComputerChoices = first(ProbOfBadComputerChoices),
    ProbOfAbsurdComputerChoices = first(ProbOfAbsurdComputerChoices),
    ComputerRiskTaking = first(ComputerRiskTaking),
    PBetChoices = first(PBet_count),
    TimeInSeconds = first(TimeInSeconds)
  ) %>%
  ungroup() %>%
  mutate(
    DelegationGroup = ifelse(sumDelegations == 0, "Keine Delegation", "Mit Delegation")
  )
analysis_data$Female2 <- analysis_data$Female
analysis_data$Female2[analysis_data$Female < 0 | analysis_data$Female == 0.5] <- NA
analysis_data$HHIncome2 <- analysis_data$HHIncome
analysis_data$HHIncome2[analysis_data$HHIncome < 0] <- NA
```

```r
analysis_data$HHIncomeLow <- 1*(analysis_data$HHIncome <= 45)
analysis_data$EduLow <- 1*(analysis_data$Education <= 3)
analysis_data$NonDelegator <- 1*(analysis_data$DelegationGroup == "Keine Delegation")
analysis_data$DelegatorType <- if_else(analysis_data$sumDelegations == 0,
↪  "NeverDelegator", if_else(analysis_data$sumDelegations == 8, "AlwaysDelegator",
↪  "SometimesDelegator"))
analysis_data$DelegatorType <- factor(analysis_data$DelegatorType, levels =
↪  c("NeverDelegator", "SometimesDelegator", "AlwaysDelegator"))

prop.table(table(analysis_data$DelegatorType, analysis_data$EVAlgorithm),2)
```

```
##
##                              0          1
##   NeverDelegator     0.15652174 0.03389831
##   SometimesDelegator 0.65217391 0.83050847
##   AlwaysDelegator    0.19130435 0.13559322
```

```r
t.test(TimeInSeconds~NonDelegator, analysis_data) # 1113 (SometimesOrAlways) vs 1237
↪  (NonDelegators) seconds
```

```
##
##  Welch Two Sample t-test
##
## data:  TimeInSeconds by NonDelegator
## t = -0.9713, df = 25.374, p-value = 0.3406
## alternative hypothesis: true difference in means between group 0 and group 1 is not equal to 0
## 95 percent confidence interval:
##  -385.8898  138.4297
## sample estimates:
## mean in group 0 mean in group 1
##        1113.043        1236.773
```

```r
analysis_data <- analysis_data %>%
  left_join(
    subjects %>% select(ProlificID, RiskTaking, FinMarketExperience,
↪   FinanciallyResponsible, AIXperience),
    by = "ProlificID"
  )

summary(lm(NonDelegator~Female2+Age+HHIncomeLow*HHIncome2 + EduLow*Education +
↪  PBetChoices + RiskTaking, data = analysis_data))
```

```
##
## Call:
## lm(formula = NonDelegator ~ Female2 + Age + HHIncomeLow * HHIncome2 +
##     EduLow * Education + PBetChoices + RiskTaking, data = analysis_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.16548 -0.11676 -0.08292 -0.04092  0.96046
##
## Coefficients:
##                         Estimate Std. Error t value Pr(>|t|)
## (Intercept)            0.1265267  0.2531926   0.500    0.618
```

```
## Female2                -0.0744037  0.0432615  -1.720    0.087 .
## Age                    -0.0004193  0.0015129  -0.277    0.782
## HHIncomeLow            -0.0045967  0.1537838  -0.030    0.976
## HHIncome2              -0.0003181  0.0016656  -0.191    0.849
## EduLow                 -0.0017206  0.2220343  -0.008    0.994
## Education               0.0084008  0.0465009   0.181    0.857
## PBetChoices             0.0027247  0.0081668   0.334    0.739
## RiskTaking             -0.0008575  0.0077844  -0.110    0.912
## HHIncomeLow:HHIncome2   0.0001018  0.0029240   0.035    0.972
## EduLow:Education       -0.0140456  0.0621721  -0.226    0.821
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2938 on 206 degrees of freedom
##   (16 Beobachtungen als fehlend gelöscht)
## Multiple R-squared:  0.02081,    Adjusted R-squared:  -0.02673
## F-statistic: 0.4377 on 10 and 206 DF,  p-value: 0.9267
```

```
summary(lm(NonDelegator~ProbOfBadComputerChoices + ProbOfAbsurdComputerChoices +
↪  Female2+Age+HHIncomeLow*HHIncome2 + EduLow*Education + PBetChoices, data =
↪  analysis_data))
```

```
##
## Call:
## lm(formula = NonDelegator ~ ProbOfBadComputerChoices + ProbOfAbsurdComputerChoices +
##     Female2 + Age + HHIncomeLow * HHIncome2 + EduLow * Education +
##     PBetChoices, data = analysis_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.19739 -0.11954 -0.08838 -0.04181  0.94212
##
## Coefficients:
##                               Estimate Std. Error t value Pr(>|t|)
## (Intercept)                  0.1523562  0.2555645   0.596   0.5517
## ProbOfBadComputerChoices    -0.0083590  0.0095685  -0.874   0.3834
## ProbOfAbsurdComputerChoices  0.0103442  0.0131243   0.788   0.4315
## Female2                     -0.0733810  0.0413246  -1.776   0.0773 .
## Age                         -0.0003151  0.0014950  -0.211   0.8333
## HHIncomeLow                 -0.0085606  0.1538121  -0.056   0.9557
## HHIncome2                   -0.0003941  0.0016675  -0.236   0.8134
## EduLow                      -0.0161442  0.2224433  -0.073   0.9422
## Education                    0.0058265  0.0463670   0.126   0.9001
## PBetChoices                  0.0032056  0.0080665   0.397   0.6915
## HHIncomeLow:HHIncome2        0.0001088  0.0029158   0.037   0.9703
## EduLow:Education            -0.0122361  0.0621862  -0.197   0.8442
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2938 on 205 degrees of freedom
##   (16 Beobachtungen als fehlend gelöscht)
## Multiple R-squared:  0.02543,    Adjusted R-squared:  -0.02686
## F-statistic: 0.4863 on 11 and 205 DF,  p-value: 0.9105
```

```r
summary(lm(NonDelegator~EVAlgorithm*(ProbOfBadComputerChoices +
↪  ProbOfAbsurdComputerChoices) + Female2+Age+HHIncomeLow*HHIncome2 + EduLow*Education
↪  + PBetChoices, data = analysis_data))
```

```
##
## Call:
## lm(formula = NonDelegator ~ EVAlgorithm * (ProbOfBadComputerChoices +
##     ProbOfAbsurdComputerChoices) + Female2 + Age + HHIncomeLow *
##     HHIncome2 + EduLow * Education + PBetChoices, data = analysis_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.25194 -0.14181 -0.07092 -0.00414  0.95739
##
## Coefficients:
##                                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)                             0.2980634  0.2631354   1.133   0.2587
## EVAlgorithm                            -0.2318006  0.0897719  -2.582   0.0105
## ProbOfBadComputerChoices               -0.0104706  0.0150058  -0.698   0.4861
## ProbOfAbsurdComputerChoices            -0.0055077  0.0192479  -0.286   0.7751
## Female2                                -0.0604063  0.0407494  -1.482   0.1398
## Age                                    -0.0004698  0.0014746  -0.319   0.7504
## HHIncomeLow                            -0.0343464  0.1511179  -0.227   0.8204
## HHIncome2                              -0.0006523  0.0016400  -0.398   0.6912
## EduLow                                 -0.0169266  0.2213200  -0.076   0.9391
## Education                               0.0047970  0.0462520   0.104   0.9175
## PBetChoices                            -0.0000361  0.0080040  -0.005   0.9964
## EVAlgorithm:ProbOfBadComputerChoices    0.0164021  0.0200078   0.820   0.4133
## EVAlgorithm:ProbOfAbsurdComputerChoices  0.0259000  0.0260704   0.993   0.3217
## HHIncomeLow:HHIncome2                   0.0006167  0.0028659   0.215   0.8298
## EduLow:Education                       -0.0175112  0.0619584  -0.283   0.7778
##
## (Intercept)
## EVAlgorithm                            *
## ProbOfBadComputerChoices
## ProbOfAbsurdComputerChoices
## Female2
## Age
## HHIncomeLow
## HHIncome2
## EduLow
## Education
## PBetChoices
## EVAlgorithm:ProbOfBadComputerChoices
## EVAlgorithm:ProbOfAbsurdComputerChoices
## HHIncomeLow:HHIncome2
## EduLow:Education
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2883 on 202 degrees of freedom
##   (16 Beobachtungen als fehlend gelöscht)
## Multiple R-squared:  0.07558,    Adjusted R-squared:  0.01151
```

```
## F-statistic:  1.18 on 14 and 202 DF,  p-value: 0.2931
```

```r
summary(lm(NonDelegator~EVAlgorithm*(ProbOfBadComputerChoices +
↪  ProbOfAbsurdComputerChoices) + Female2+Age+HHIncomeLow*HHIncome2 + EduLow*Education
↪  + PBetChoices + RiskTaking + FinMarketExperience + FinanciallyResponsible +
↪  AIXperience, data = analysis_data))
```

```
##
## Call:
## lm(formula = NonDelegator ~ EVAlgorithm * (ProbOfBadComputerChoices +
##     ProbOfAbsurdComputerChoices) + Female2 + Age + HHIncomeLow *
##     HHIncome2 + EduLow * Education + PBetChoices + RiskTaking +
##     FinMarketExperience + FinanciallyResponsible + AIXperience,
##     data = analysis_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.26109 -0.13723 -0.07054 -0.00492  1.01078
##
## Coefficients:
##                                       Estimate Std. Error t value Pr(>|t|)
## (Intercept)                          0.3807180  0.2730279   1.394  0.16475
## EVAlgorithm                         -0.2457715  0.0908956  -2.704  0.00745
## ProbOfBadComputerChoices            -0.0125361  0.0151941  -0.825  0.41033
## ProbOfAbsurdComputerChoices         -0.0060350  0.0197587  -0.305  0.76035
## Female2                             -0.0597789  0.0442632  -1.351  0.17839
## Age                                 -0.0003886  0.0015211  -0.255  0.79864
## HHIncomeLow                         -0.0112014  0.1534534  -0.073  0.94188
## HHIncome2                           -0.0006578  0.0016593  -0.396  0.69219
## EduLow                              -0.0253974  0.2226676  -0.114  0.90931
## Education                            0.0025497  0.0467996   0.054  0.95661
## PBetChoices                         -0.0007404  0.0081890  -0.090  0.92805
## RiskTaking                          -0.0002967  0.0087808  -0.034  0.97308
## FinMarketExperience                  0.0043132  0.0088149   0.489  0.62516
## FinanciallyResponsible              -0.0069961  0.0065828  -1.063  0.28917
## AIXperience                         -0.0059927  0.0070015  -0.856  0.39308
## EVAlgorithm:ProbOfBadComputerChoices     0.0188655  0.0202041   0.934  0.35157
## EVAlgorithm:ProbOfAbsurdComputerChoices  0.0267467  0.0262968   1.017  0.31034
## HHIncomeLow:HHIncome2                0.0001047  0.0029338   0.036  0.97157
## EduLow:Education                    -0.0141313  0.0625078  -0.226  0.82138
##
## (Intercept)
## EVAlgorithm                         **
## ProbOfBadComputerChoices
## ProbOfAbsurdComputerChoices
## Female2
## Age
## HHIncomeLow
## HHIncome2
## EduLow
## Education
## PBetChoices
## RiskTaking
## FinMarketExperience
```

```
## FinanciallyResponsible
## AIXperience
## EVAlgorithm:ProbOfBadComputerChoices
## EVAlgorithm:ProbOfAbsurdComputerChoices
## HHIncomeLow:HHIncome2
## EduLow:Education
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2896 on 198 degrees of freedom
##   (16 Beobachtungen als fehlend gelöscht)
## Multiple R-squared:  0.08525,    Adjusted R-squared:  0.002091
## F-statistic: 1.025 on 18 and 198 DF,  p-value: 0.4332
```

```r
summary(lm(NonDelegator~EVAlgorithm*(RiskTaking + FinMarketExperience +
↪  FinanciallyResponsible + AIXperience), data = analysis_data))
```

```
##
## Call:
## lm(formula = NonDelegator ~ EVAlgorithm * (RiskTaking + FinMarketExperience +
##     FinanciallyResponsible + AIXperience), data = analysis_data)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.35029 -0.12939 -0.04450 -0.02412  0.97733
##
## Coefficients:
##                                     Estimate Std. Error t value Pr(>|t|)
## (Intercept)                         0.300200   0.073152   4.104  5.7e-05 ***
## EVAlgorithm                        -0.291137   0.106421  -2.736  0.00673 **
## RiskTaking                          0.012182   0.011139   1.094  0.27529
## FinMarketExperience                 0.013547   0.010710   1.265  0.20724
## FinanciallyResponsible             -0.023445   0.008860  -2.646  0.00872 **
## AIXperience                        -0.016839   0.009492  -1.774  0.07743 .
## EVAlgorithm:RiskTaking             -0.013997   0.015571  -0.899  0.36968
## EVAlgorithm:FinMarketExperience    -0.013368   0.015217  -0.878  0.38063
## EVAlgorithm:FinanciallyResponsible  0.027458   0.011927   2.302  0.02225 *
## EVAlgorithm:AIXperience             0.017965   0.013100   1.371  0.17163
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.2847 on 223 degrees of freedom
## Multiple R-squared:  0.09268,    Adjusted R-squared:  0.05606
## F-statistic: 2.531 on 9 and 223 DF,  p-value: 0.008803
```

```r
# 2. Deskriptive Statistik pro Gruppe
summary_stats <- analysis_data %>%
  group_by(DelegationGroup) %>%
  summarise(
    MeanAge = mean(Age, na.rm = TRUE),
    SDAge = sd(Age, na.rm = TRUE),
    PropFemale = mean(Female, na.rm = TRUE),
    MeanBadComputerChoices = mean(ProbOfBadComputerChoices, na.rm = TRUE),
    MeanAbsurdComputerChoices = mean(ProbOfAbsurdComputerChoices, na.rm = TRUE),
    MeanComputerRiskTaking = mean(ComputerRiskTaking, na.rm = TRUE),
```

```r
    MeanPBetChoices = mean(PBetChoices, na.rm = TRUE),
    N = n()
  )

print(summary_stats)
```

```
## # A tibble: 2 x 9
##   DelegationGroup  MeanAge SDAge PropFemale MeanBadComputerChoices
##   <chr>              <dbl> <dbl>      <dbl>                  <dbl>
## 1 Keine Delegation    41.8  14.8      0.341                   4.36
## 2 Mit Delegation      42.9  13.6      0.498                   4.40
## # i 4 more variables: MeanAbsurdComputerChoices <dbl>,
## #   MeanComputerRiskTaking <dbl>, MeanPBetChoices <dbl>, N <int>
```

```r
# 3. Statistische Tests pro Variable
# a) Female (binär)
# fisher.test(table(analysis_data$DelegationGroup, analysis_data$Female))

# Kontingenztabelle erstellen
female_table <- table(
  DelegationGroup = analysis_data$DelegationGroup,
  Female = analysis_data$Female
)
prop.table(female_table,2)[,c(2,4)]
```

```
##                   Female
## DelegationGroup            0         1
##   Keine Delegation 0.1196581 0.0625000
##   Mit Delegation   0.8803419 0.9375000
```

```r
exact.test(female_table[,c(2,4)], method = "Boschloo", to.plot = F)$p.value
```

```
## [1] 0.1561955
```

```r
# b) Age (numerisch)
t.test(Age ~ DelegationGroup, data = analysis_data)
```

```
##
##  Welch Two Sample t-test
##
## data:  Age by DelegationGroup
## t = -0.32876, df = 24.878, p-value = 0.7451
## alternative hypothesis: true difference in means between group Keine Delegation and group Mit Delega
## 95 percent confidence interval:
##  -7.849952  5.689245
## sample estimates:
## mean in group Keine Delegation   mean in group Mit Delegation
##                       41.77273                       42.85308
```

```r
# c) HHIncome (kategorial)
table(analysis_data$DelegationGroup, analysis_data$HHIncome)
```

```
##
##                    -1  5 15 25 35 45 55 65 75 85 95 105
##   Keine Delegation  1  1  1  5  5  1  2  1  3  0  0   2
```

```
##   Mit Delegation   12  4 22 33 35 26 19 20  5 10  7  18
```

```r
chisq.test(table(analysis_data$DelegationGroup, analysis_data$HHIncome))
```

```
## Warning in chisq.test(table(analysis_data$DelegationGroup,
## analysis_data$HHIncome)): Chi-Quadrat-Approximation kann inkorrekt sein

##
##  Pearson's Chi-squared test
##
## data:  table(analysis_data$DelegationGroup, analysis_data$HHIncome)
## X-squared = 13.171, df = 11, p-value = 0.2823
```

```r
# d) Education (kategorial)
table(analysis_data$DelegationGroup, analysis_data$Education)
```

```
##
##                      1  2  3  4  5  6
##   Keine Delegation  2  4  1 10  4  1
##   Mit Delegation   23 37 29 85 31  6
```

```r
chisq.test(table(analysis_data$DelegationGroup, analysis_data$Education))
```

```
## Warning in chisq.test(table(analysis_data$DelegationGroup,
## analysis_data$Education)): Chi-Quadrat-Approximation kann inkorrekt sein

##
##  Pearson's Chi-squared test
##
## data:  table(analysis_data$DelegationGroup, analysis_data$Education)
## X-squared = 1.859, df = 5, p-value = 0.8683
```

```r
# e) BadComputerChoices (numerisch)
t.test(ProbOfBadComputerChoices ~ DelegationGroup, data = analysis_data)
```

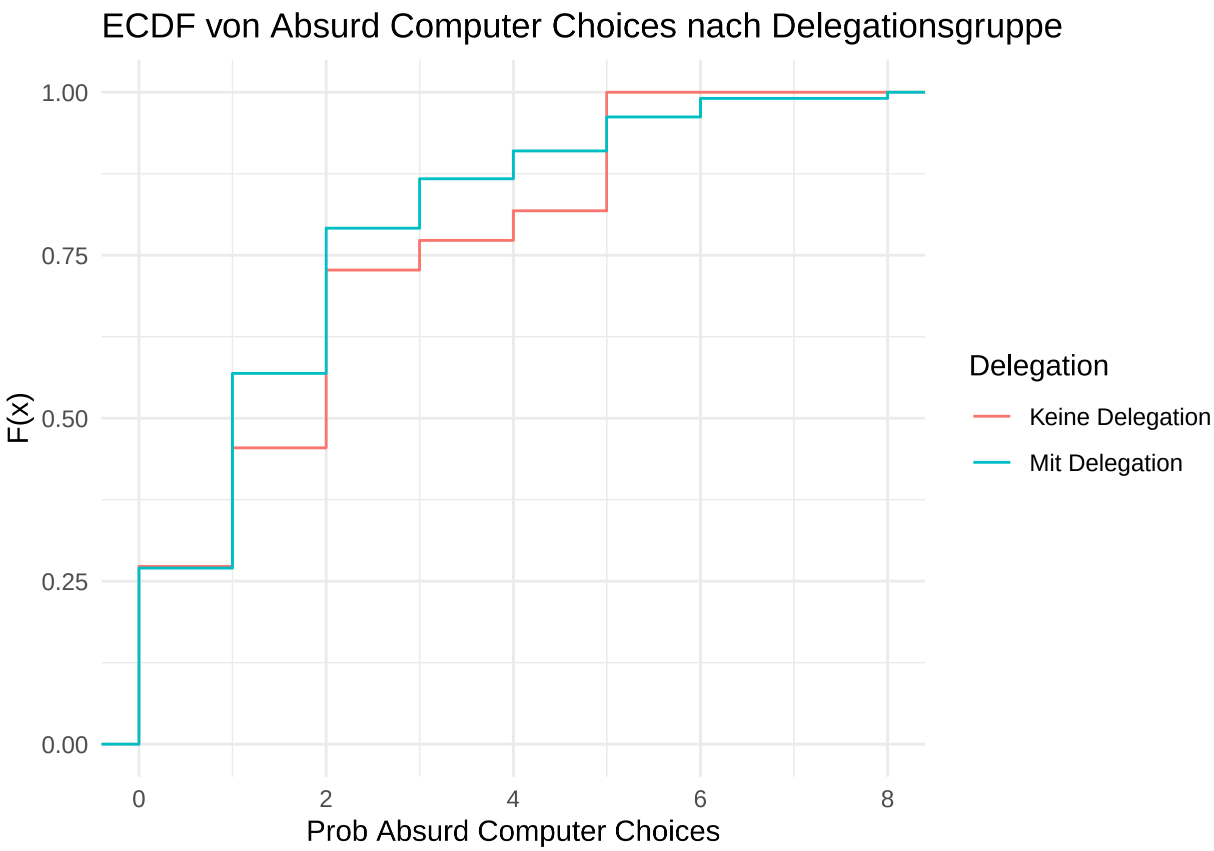
```
##
##  Welch Two Sample t-test
##
## data:  ProbOfBadComputerChoices by DelegationGroup
## t = -0.073675, df = 25.323, p-value = 0.9418
## alternative hypothesis: true difference in means between group Keine Delegation and group Mit Delega
## 95 percent confidence interval:
##  -1.134517  1.056102
## sample estimates:
## mean in group Keine Delegation   mean in group Mit Delegation
##                        4.363636                       4.402844
```

```r
# f) AbsurdComputerChoices (numerisch)
t.test(ProbOfAbsurdComputerChoices ~ DelegationGroup, data = analysis_data)
```

```
##
##  Welch Two Sample t-test
##
## data:  ProbOfAbsurdComputerChoices by DelegationGroup
## t = 0.7574, df = 24.854, p-value = 0.4559
## alternative hypothesis: true difference in means between group Keine Delegation and group Mit Delega
## 95 percent confidence interval:
```

```
##  -0.5250476  1.1355603
## sample estimates:
## mean in group Keine Delegation   mean in group Mit Delegation
##                       1.954545                       1.649289
```

```
# 1.95 vs 1.649, p = 0.4559
ggplot(analysis_data, aes(x = ProbOfAbsurdComputerChoices, color = DelegationGroup)) +
  stat_ecdf(geom = "step") +
  labs(title = "ECDF von Absurd Computer Choices nach Delegationsgruppe",
       x = "Prob Absurd Computer Choices",
       y = "F(x)",
       color = "Delegation") +
  theme_minimal()
```



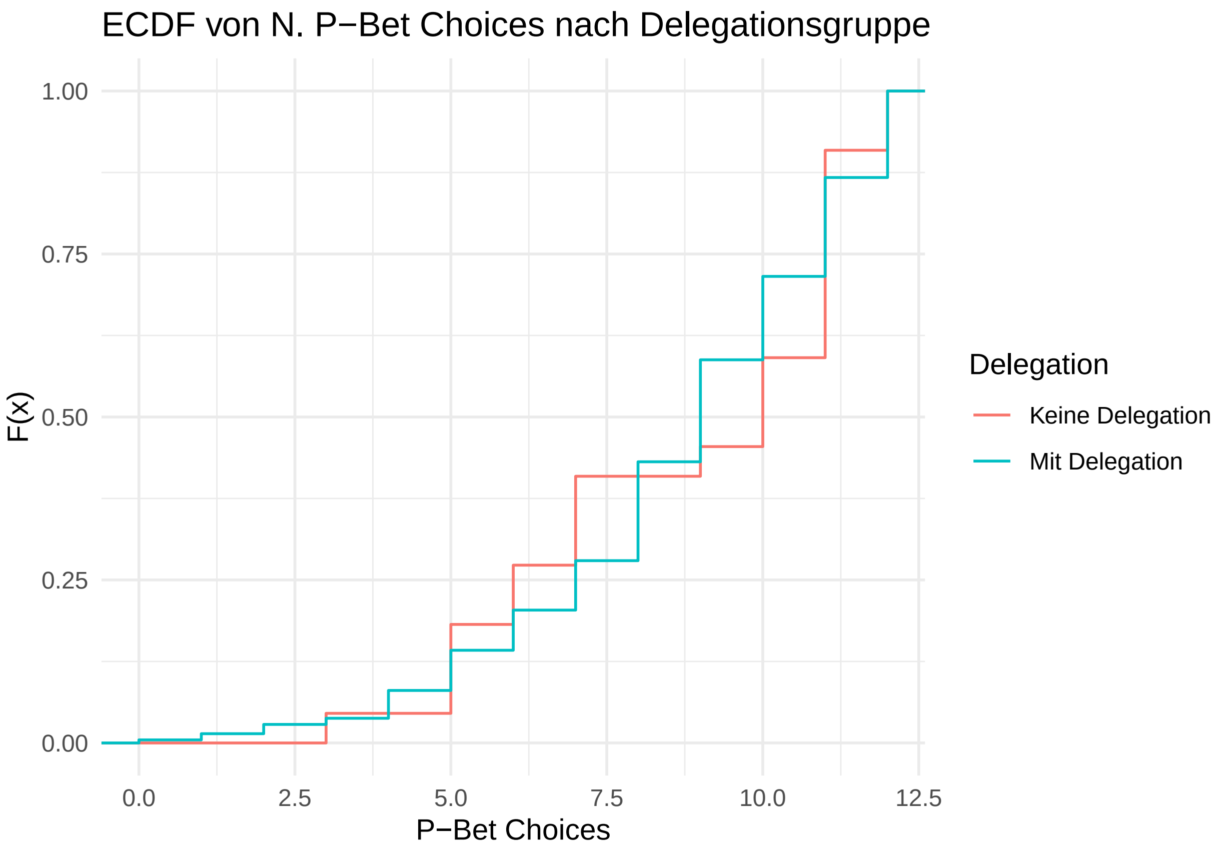

```
ks.test(
  as.numeric(analysis_data$ProbOfAbsurdComputerChoices[analysis_data$DelegationGroup ==
  ↪  "Keine Delegation"]),
  as.numeric(analysis_data$ProbOfAbsurdComputerChoices[analysis_data$DelegationGroup ==
  ↪  "Mit Delegation"])
) # p = 0.5876
```

```
##
##  Exact two-sample Kolmogorov-Smirnov test
##
## data:  as.numeric(analysis_data$ProbOfAbsurdComputerChoices[analysis_data$DelegationGroup == "Keine I
## D = 0.11417, p-value = 0.5876
## alternative hypothesis: two-sided
```

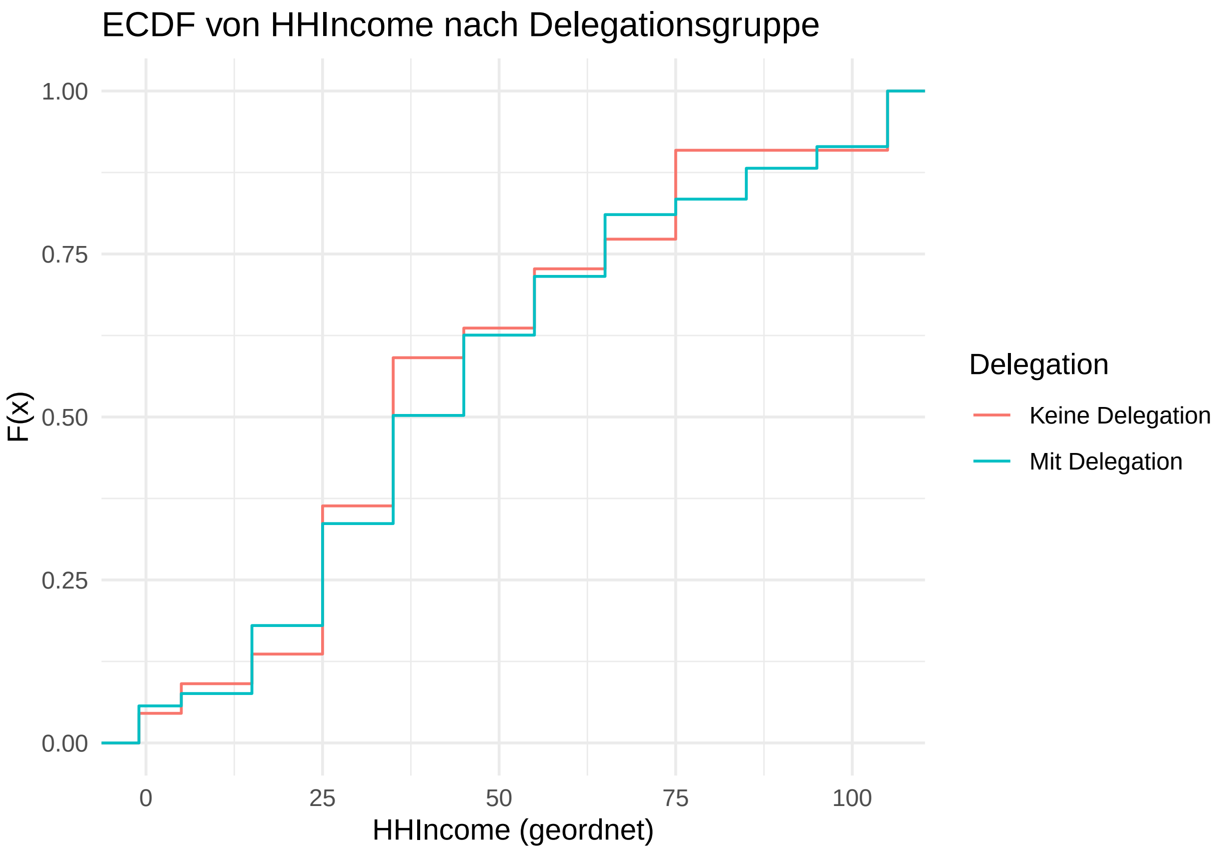
```r
choices %>%
  distinct(ProlificID, .keep_all = TRUE) %>%
  mutate(
    sumDelegations_group = ifelse(sumDelegations == 0, "sumDelegations = 0",
     ↪  "sumDelegations > 0"),
    hasPositiveProb = ProbOfBadComputerChoices > 0
  ) %>%
  group_by(sumDelegations_group, hasPositiveProb) %>%
  summarise(Anzahl = n(), .groups = "drop") %>%
  pivot_wider(
    names_from = hasPositiveProb,
    values_from = Anzahl,
    names_prefix = "Prob_"
  ) %>%
  mutate(
    Total = Prob_TRUE + Prob_FALSE,
    Prob_TRUE_pct = round(Prob_TRUE / Total * 100, 1),
    Prob_FALSE_pct = round(Prob_FALSE / Total * 100, 1)
  ) %>%
  select(-Total, -Prob_TRUE, -Prob_FALSE)
```

```
## # A tibble: 2 x 3
##   sumDelegations_group Prob_TRUE_pct Prob_FALSE_pct
##   <chr>                        <dbl>          <dbl>
## 1 sumDelegations = 0            95.5            4.5
## 2 sumDelegations > 0            93.4            6.6
```

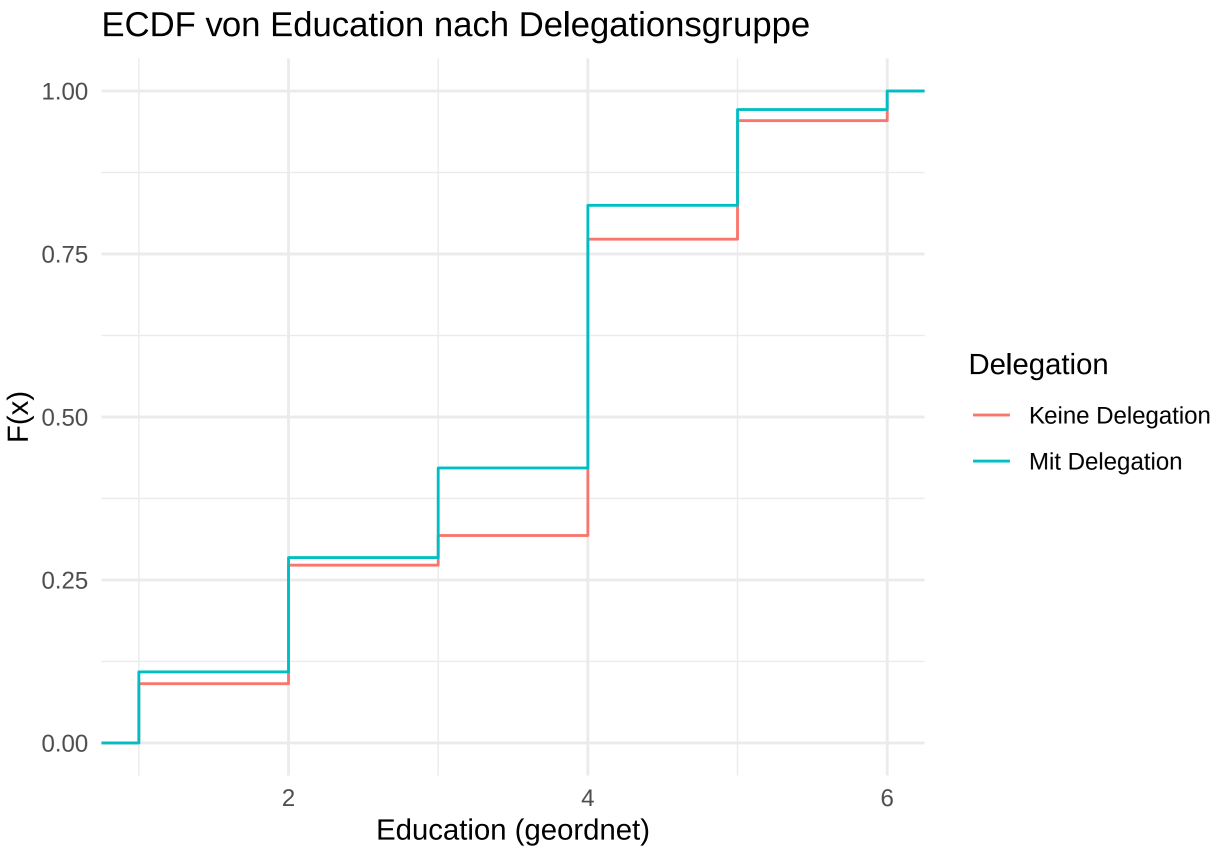
```r
ggplot(analysis_data, aes(x = PBetChoices, color = DelegationGroup)) +
  stat_ecdf(geom = "step") +
  labs(title = "ECDF von N. P-Bet Choices nach Delegationsgruppe",
       x = "P-Bet Choices",
       y = "F(x)",
       color = "Delegation") +
  theme_minimal()
```




```r
# 1. HHIncome
ggplot(analysis_data, aes(x = HHIncome, color = DelegationGroup)) +
  stat_ecdf(geom = "step") +
  labs(title = "ECDF von HHIncome nach Delegationsgruppe",
       x = "HHIncome (geordnet)",
       y = "F(x)",
       color = "Delegation") +
  theme_minimal()
```




```r
# 2. Education
ggplot(analysis_data, aes(x = Education, color = DelegationGroup)) +
  stat_ecdf(geom = "step") +
  labs(title = "ECDF von Education nach Delegationsgruppe",
       x = "Education (geordnet)",
       y = "F(x)",
       color = "Delegation") +
  theme_minimal()
```




```r
# KS-Test für HHIncome
ks.test(
  as.numeric(analysis_data$HHIncome[analysis_data$DelegationGroup == "Keine
  ↪  Delegation"]),
  as.numeric(analysis_data$HHIncome[analysis_data$DelegationGroup == "Mit Delegation"])
)
```

```
##
##  Exact two-sample Kolmogorov-Smirnov test
##
## data:  as.numeric(analysis_data$HHIncome[analysis_data$DelegationGroup == "Keine Delegation"]) and a
## D = 0.088539, p-value = 0.9099
## alternative hypothesis: two-sided
```

```r
# KS-Test für Education
ks.test(
  as.numeric(analysis_data$Education[analysis_data$DelegationGroup == "Keine
  ↪  Delegation"]),
  as.numeric(analysis_data$Education[analysis_data$DelegationGroup == "Mit Delegation"])
)
```

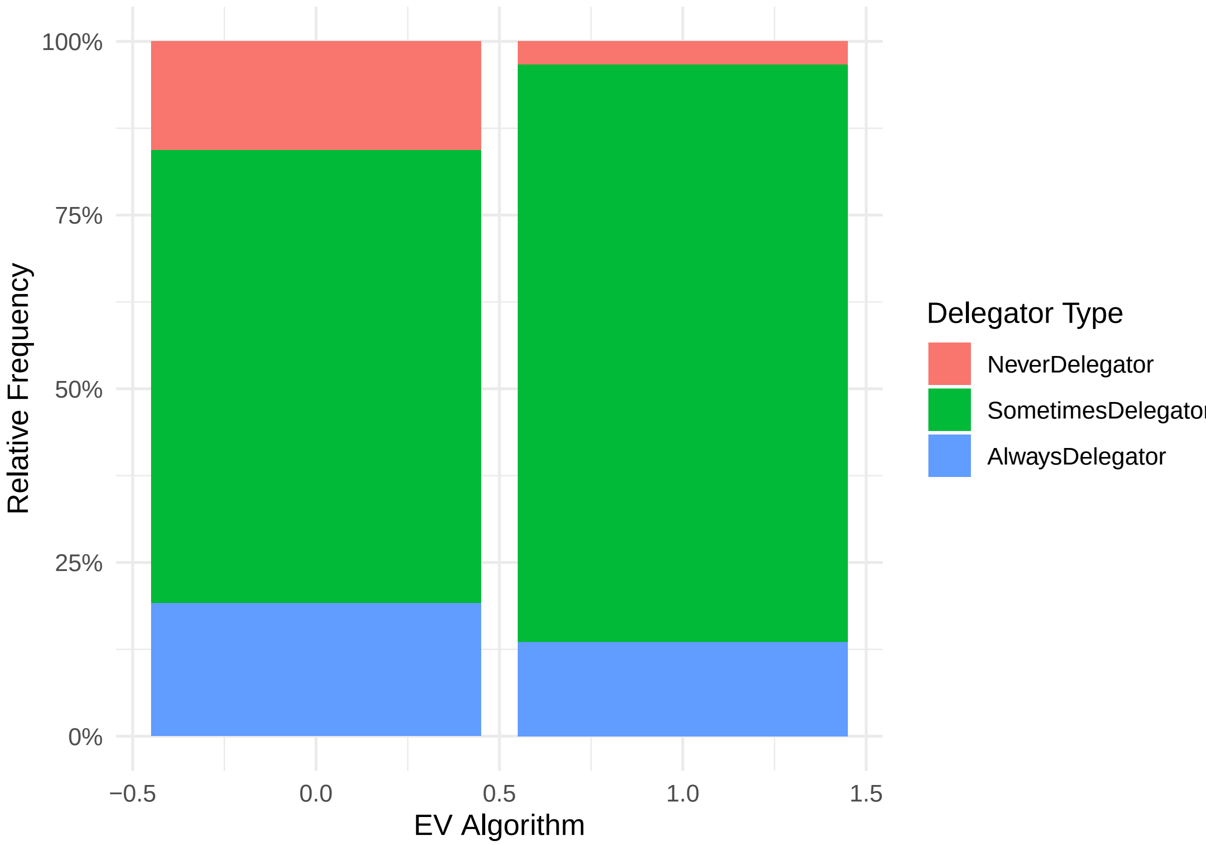
```
##
##  Exact two-sample Kolmogorov-Smirnov test
##
## data:  as.numeric(analysis_data$Education[analysis_data$DelegationGroup == "Keine Delegation"]) and a
## D = 0.10362, p-value = 0.6007
## alternative hypothesis: two-sided
```

```r
# Step 1: Filter for non-delegated choices and calculate frequency of non-dominated
↪  choices
nonDelegatedChoices <- choices %>%
  filter(Delegated == 0, Option > 0) %>%
  group_by(ProlificID, Algorithm) %>%
  summarise(
    nonDomCount = sum(Nondominated, na.rm = TRUE),
    totalNonDelegated = n(),
    .groups = "drop"
  ) %>%
  mutate(freqNonDom = nonDomCount / totalNonDelegated)

# Step 2: Join with delegation status
nonDelegatedChoices <- nonDelegatedChoices %>%
  left_join(
    choices %>%
      select(ProlificID, sumDelegations) %>%
      distinct(),
    by = "ProlificID"
  )

# Step 3: Group by delegation status and calculate mean frequency
nonDelegatedChoices %>%
  group_by(sumDelegations == 0, Algorithm) %>%
  summarise(
    meanFreqNonDom = mean(freqNonDom, na.rm = TRUE),
    sdFreqNonDom = sd(freqNonDom, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  ) %>%
  mutate(
    delegationGroup = ifelse(`sumDelegations == 0`, "Never-delegators",
    ↪  "Sometimes-delegators")
  ) %>%
  select(delegationGroup, Algorithm, meanFreqNonDom, sdFreqNonDom, n)
```

```
## # A tibble: 4 x 5
##   delegationGroup      Algorithm meanFreqNonDom sdFreqNonDom     n
##   <chr>                <chr>              <dbl>        <dbl> <int>
## 1 Sometimes-delegators EV                 0.340        0.249    96
## 2 Sometimes-delegators Pref               0.332        0.271    74
## 3 Never-delegators     EV                 0.312        0.161     4
## 4 Never-delegators     Pref               0.354        0.178    18
```

```r
plot_data <- analysis_data %>%
  group_by(EVAlgorithm, DelegatorType) %>%
  summarise(n = n(), .groups = "drop") %>%
  group_by(EVAlgorithm) %>%
  mutate(rel_freq = n / sum(n))

plot_data <- plot_data %>%
  mutate(
    DelegatorType = factor(DelegatorType,
                           levels = c("AlwaysDelegator", "SometimesDelegator",
                           ↪  "NeverDelegator"))
```

```
  )

ggplot(analysis_data, aes(x = EVAlgorithm, fill = DelegatorType)) +
  geom_bar(position = "fill") +
  labs(
    x = "EV Algorithm",
    y = "Relative Frequency",
    fill = "Delegator Type"
  ) +
  scale_y_continuous(labels = scales::percent_format()) +
  theme_minimal()
```




```
plot_DelTypes <- ggplot(plot_data, aes(x = factor(EVAlgorithm), y = rel_freq, fill =
↪  factor(DelegatorType, levels = c("NeverDelegator", "SometimesDelegator",
↪  "AlwaysDelegator")))) +
  labs(
    title = "Delegator Types by Algorithm",
    x = "EV Algorithm",
    y = "Relative Frequency",
    fill = "Delegator Type"
  ) +
  geom_bar(stat = "identity", position = "dodge") +
  scale_y_continuous(labels = scales::percent_format()) +
#  scale_fill_brewer(palette = "Set2") +
  theme_minimal()
```

```r
ggsave("DelegatorTypesByTrmt.png",
       plot = plot_DelTypes,
       width = 6, height = 4,   # in inches
       dpi = 300)
```

win-stay, lose-shift

```r
choices <- choices %>%
  group_by(ProlificID) %>%
  arrange(WithinPartDecisionNumber, .by_group = TRUE) %>%
  mutate(
    lagDelegated          = lag(Delegated, 1),
    lagRelevantPayoff     = lag(RelevantPayoff, 1),
    lagSatisfactionRating = lag(SatisfactionRating, 1)
  ) %>%
  ungroup()

choices$stay <- 1*(choices$Delegated == choices$lagDelegated)
choices$lagWin <- 1*(choices$RelevantPayoff > 0)

winStayReg <- feols(
  stay ~ EVAlgorithm*lagWin | OptionsetID + WithinPartDecisionNumber,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2 & WithinPartDecisionNumber > 1)
)
```

```
## NOTE: 2 observations removed because of NA values (LHS: 2).
```

```r
summary(winStayReg)
```

```
## OLS estimation, Dep. Var.: stay
## Observations: 1,615
## Fixed-effects: OptionsetID: 8,  WithinPartDecisionNumber: 7
## Standard-errors: Clustered (ProlificID)
##                     Estimate Std. Error   t value Pr(>|t|)
## EVAlgorithm        -0.038714   0.046728 -0.828502 0.408238
## lagWin              0.042733   0.032822  1.301938 0.194229
## EVAlgorithm:lagWin -0.101566   0.049417 -2.055308 0.040969 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.463812     Adj. R2: 0.023765
##                  Within R2: 0.014698
```

```r
winStayReg10 <- feols(
  stay ~ lagDelegated*I(1-lagWin) | OptionsetID + WithinPartDecisionNumber,
  cluster = ~ProlificID,
  data = subset(choices, EVAlgorithm == 1 & Part == 2 & WithinPartDecisionNumber > 1)
)
summary(winStayReg10)
```

```
## OLS estimation, Dep. Var.: stay
## Observations: 819
## Fixed-effects: OptionsetID: 8,  WithinPartDecisionNumber: 7
## Standard-errors: Clustered (ProlificID)
```

```
##                           Estimate Std. Error   t value Pr(>|t|)
## lagDelegated             -0.084825   0.060180 -1.409503  0.16134
## I(1 - lagWin)             0.001031   0.055659  0.018526  0.98525
## lagDelegated:I(1 - lagWin)  0.117426   0.072470  1.620344  0.10785
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.480487     Adj. R2: 0.005125
##                  Within R2: 0.008513
```

```
winStayReg20 <- feols(
  stay ~ lagDelegated*I(1-lagWin) | OptionsetID + WithinPartDecisionNumber,
  cluster = ~ProlificID,
  data = subset(choices, EVAlgorithm == 0 & Part == 2 & WithinPartDecisionNumber > 1)
)
```

```
## NOTE: 2 observations removed because of NA values (LHS: 2, RHS: 2).
```

```
summary(winStayReg20)
```

```
## OLS estimation, Dep. Var.: stay
## Observations: 796
## Fixed-effects: OptionsetID: 8,  WithinPartDecisionNumber: 7
## Standard-errors: Clustered (ProlificID)
##                           Estimate Std. Error   t value Pr(>|t|)
## lagDelegated             -0.002146   0.050853 -0.042207  0.96641
## I(1 - lagWin)            -0.015209   0.043524 -0.349451  0.72740
## lagDelegated:I(1 - lagWin) -0.060019   0.067862 -0.884420  0.37833
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.441256     Adj. R2: 0.020019
##                  Within R2: 0.00407
```

```
winStayReg2 <- feols(
  stay ~ EVAlgorithm*lagSatisfactionRating | OptionsetID + WithinPartDecisionNumber,
  cluster = ~ProlificID,
  data = subset(choices, Part == 2 & WithinPartDecisionNumber > 1)
)
```

```
## NOTE: 2 observations removed because of NA values (LHS: 2, RHS: 2).
```

```
summary(winStayReg2)
```

```
## OLS estimation, Dep. Var.: stay
## Observations: 1,615
## Fixed-effects: OptionsetID: 8,  WithinPartDecisionNumber: 7
## Standard-errors: Clustered (ProlificID)
##                                   Estimate Std. Error   t value Pr(>|t|)
## EVAlgorithm                      -0.091518   0.046542 -1.966338 0.050452 .
## lagSatisfactionRating             0.005389   0.008098  0.665476 0.506407
## EVAlgorithm:lagSatisfactionRating -0.001945   0.008211 -0.236832 0.812996
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## RMSE: 0.463585     Adj. R2: 0.024721
##                  Within R2: 0.015662
```

```r
choices %>%
  group_by(EVAlgorithm, lagDelegated, lagWin) %>%
  summarise(
    stayRate = mean(stay == 1, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  )
```

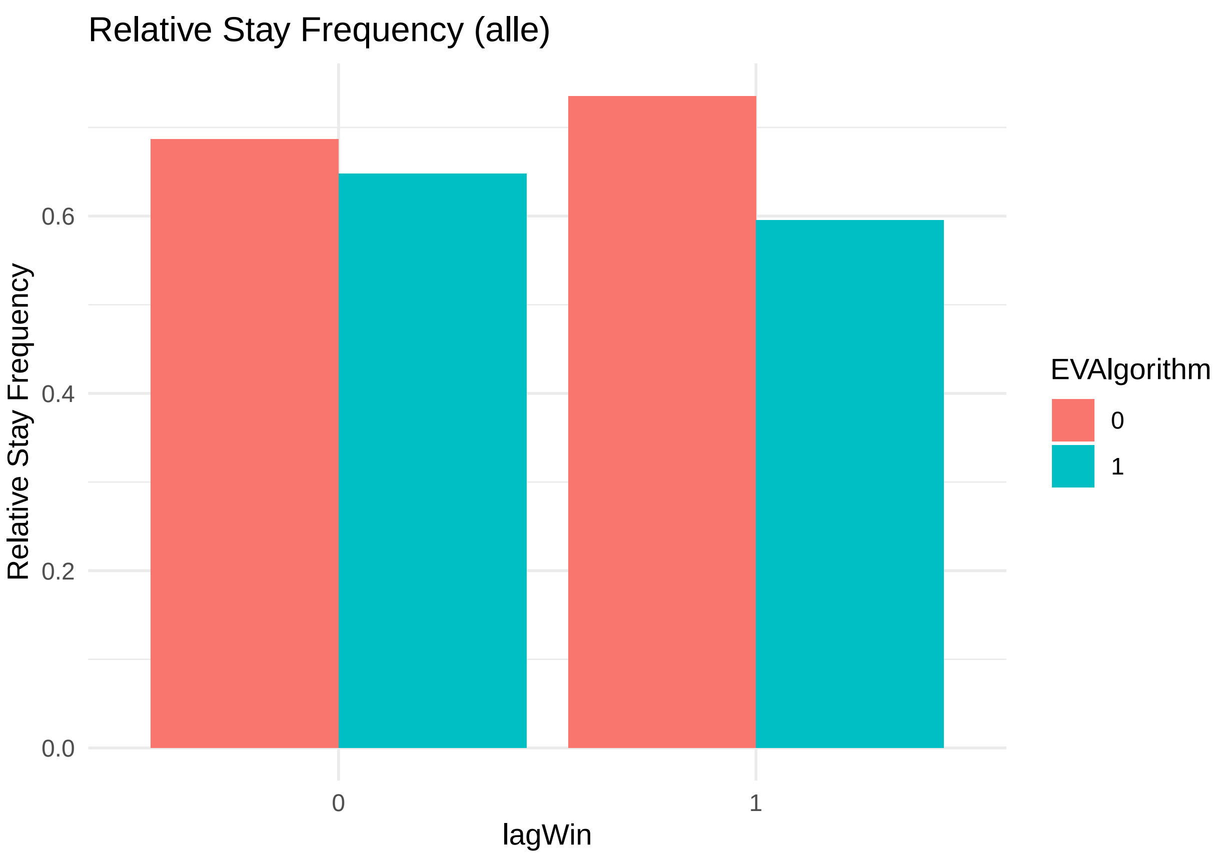
```
## # A tibble: 12 x 5
##    EVAlgorithm lagDelegated lagWin stayRate     n
##          <int>        <int>  <dbl>    <dbl> <int>
##  1           0            0      0    0.725   149
##  2           0            0      1    0.739   261
##  3           0            1      0    0.649   148
##  4           0            1      1    0.731   238
##  5           0           NA      0  NaN        38
##  6           0           NA      1  NaN        77
##  7           1            0      0    0.633   147
##  8           1            0      1    0.641   259
##  9           1            1      0    0.661   168
## 10           1            1      1    0.547   245
## 11           1           NA      0  NaN        39
## 12           1           NA      1  NaN        79
```

```r
choices %>%
  filter((WithinPartDecisionNumber == 2)) %>%
  group_by(EVAlgorithm, lagDelegated, lagWin) %>%
  summarise(
    stayRate = mean(stay == 1, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  )
```

```
## # A tibble: 10 x 5
##    EVAlgorithm lagDelegated lagWin stayRate     n
##          <int>        <int>  <dbl>    <dbl> <int>
##  1           0            0      0    0.462    13
##  2           0            0      1    0.622    37
##  3           0            1      0    0.739    23
##  4           0            1      1    0.590    39
##  5           0           NA      0  NaN         1
##  6           0           NA      1  NaN         1
##  7           1            0      0    0.5      14
##  8           1            0      1    0.56     25
##  9           1            1      0    0.645    31
## 10           1            1      1    0.5      48
```

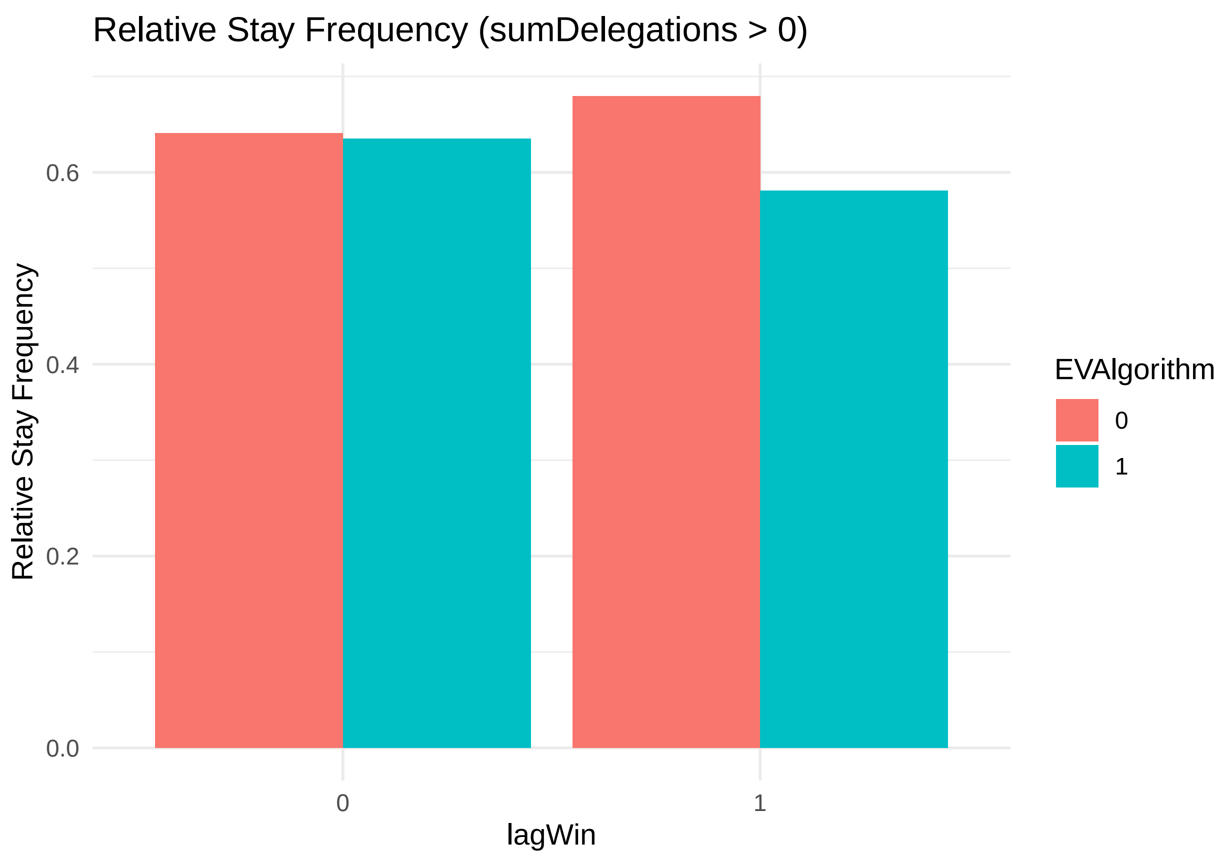
```r
# Basis-Daten (Filter)
df_base <- choices %>%
  filter(WithinPartDecisionNumber > 1)

# Funktion zur Berechnung relativer Stay-Frequenzen
calc_rel_stay <- function(df) {
  df %>%
```

```
    group_by(EVAlgorithm, lagWin) %>%
    summarise(rel_stay = mean(stay, na.rm = TRUE),
              n = n(),
              .groups = "drop")
}

# Datensätze
df_all <- calc_rel_stay(df_base)

df_deleg <- df_base %>%
  filter(sumDelegations > 0) %>%
  calc_rel_stay()

p1 <- ggplot(df_all, aes(x = factor(lagWin), y = rel_stay, fill =
↪  as.factor(EVAlgorithm))) +
  geom_col(position = "dodge") +
  labs(title = "Relative Stay Frequency (alle)",
       x = "lagWin",
       y = "Relative Stay Frequency",
       fill = "EVAlgorithm") +
  theme_minimal()

# Plot 2: Nur sumDelegations > 0 (Balken)
p2 <- ggplot(df_deleg, aes(x = factor(lagWin), y = rel_stay, fill =
↪  as.factor(EVAlgorithm))) +
  geom_col(position = "dodge") +
  labs(title = "Relative Stay Frequency (sumDelegations > 0)",
       x = "lagWin",
       y = "Relative Stay Frequency",
       fill = "EVAlgorithm") +
  theme_minimal()

# Anzeigen
p1
```



Relative Stay Frequency (alle)
0.0
0.2
0.4
0.6
Relative Stay Frequency
0
1
lagWin
EVAlgorithm
0
1


```
p2
```



with the EVAlgorithm, I switch much more after a “win” than with the PrefAlgorithm (I generally switch less [n.sign.] in EV, put have a slight tendency [n.sign.] to switch /more/ often after wins; that is reversed by the [sign.] interaction term under Pref) looking at satisfaction: when I’m dissatisfied, I switch less with the EVAlgorithm (p = 0.054; this is like last time); having said that, satisfaction doesn’t do much at all

hypothetic choices, never delegate, and reasons

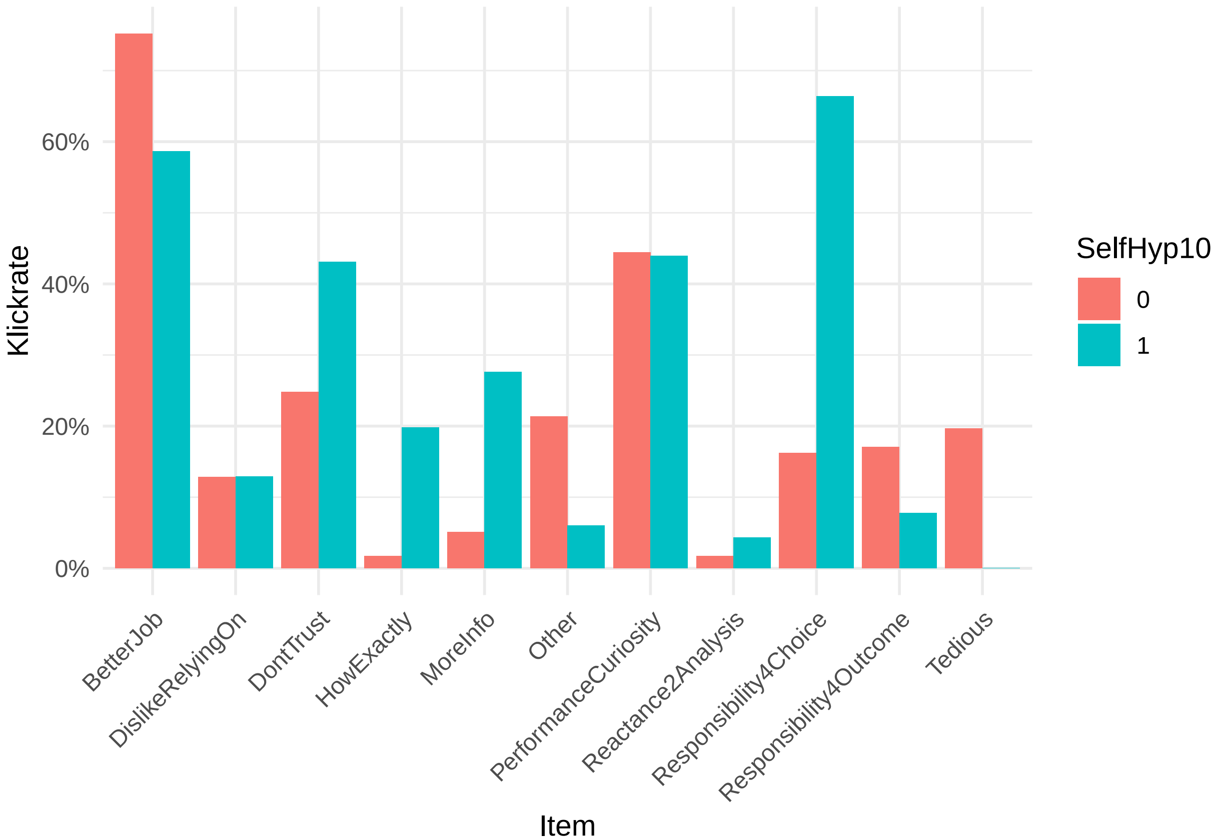
```
NeverDelegate <- choices %>%
  select(ProlificID, sumDelegations) %>%
  distinct() %>%  # eine Zeile pro Person
  mutate(sumDeleg0 = (sumDelegations == 0))

subjects2 <- subjects %>%
  left_join(NeverDelegate, by = "ProlificID")

subjects2 %>%
  group_by(EVAlgorithm, sumDeleg0) %>%
  summarise(
    SatisfiedPart1 = mean(SatisfiedPart1, na.rm = TRUE),
    n = n(),
    .groups = "drop"
  )
```

```
## # A tibble: 4 x 4
##   EVAlgorithm sumDeleg0 SatisfiedPart1     n
##         <int> <lgl>              <dbl> <int>
## 1           0 FALSE               7.56    97
```

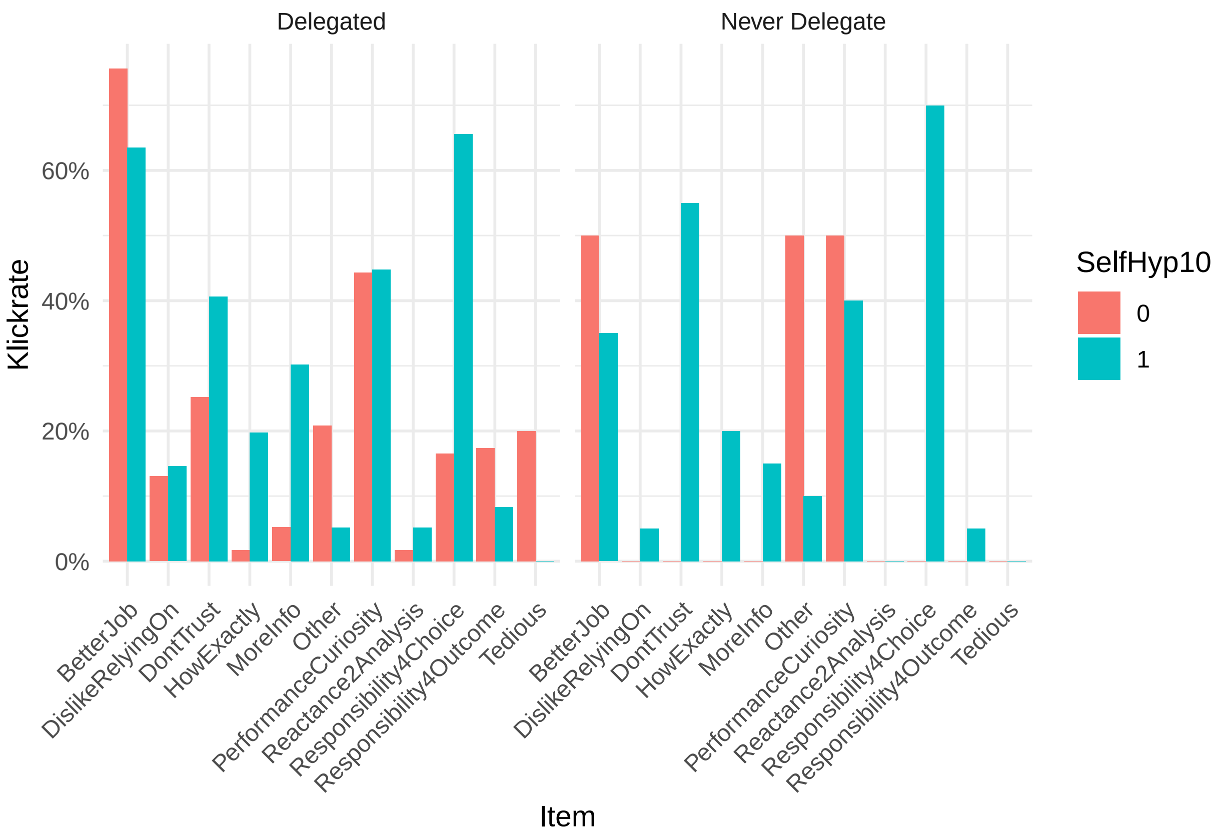
```
## 2           0 TRUE                7.94    18
## 3           1 FALSE               7.56   114
## 4           1 TRUE                8        4
```

```
subjects2$SelfHyp10 <- 1-subjects2$DelegateHyp10

subjects_long <- subjects2 %>%
  pivot_longer(
    cols = DontTrust:Other,
    names_to = "item",
    values_to = "clicked"
  )

click_rates_sumDeleg0 <- subjects_long %>%
  group_by(sumDeleg0, item) %>%
  summarise(
    rate = mean(clicked, na.rm = TRUE),
    n = n_distinct(ProlificID),
    .groups = "drop"
  )

fig47 <- ggplot(click_rates_sumDeleg0, aes(x = item, y = rate, fill = factor(sumDeleg0)))
↪  +
  # geom_col(fill = "steelblue") +
  geom_col(position = "dodge") +
  labs(x = "Item",
       y = "Rel. frecuency",
       fill = "Never Delegate",
       title = "Reasons (aggregated)") +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  # facet_wrap(~ sumDeleg0, labeller = labeller(sumDeleg0 = c(`TRUE` = "Never Delegate",
  ↪  `FALSE` = "Delegated"))) +
  theme_minimal() +
  theme(legend.position = "none") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

ggsave("ReasonsByNeverDelegator.png",
       plot = fig47,
       width = 6, height = 4,   # in inches
       dpi = 300)


person_level <- subjects_long %>%
  group_by(ProlificID, EVAlgorithm, sumDeleg0, item) %>%
  summarise(
    clicked_person = mean(clicked, na.rm = TRUE),
    .groups = "drop"
  )
click_rates_algo <- person_level %>%
  group_by(EVAlgorithm, sumDeleg0, item) %>%
  summarise(
    rate = mean(clicked_person, na.rm = TRUE),
    n = n(),
    .groups = "drop"
```

```r
  )


fig46 <- ggplot(click_rates_algo,
       aes(x = item, y = rate, fill = factor(sumDeleg0))) +
  geom_col(position = "dodge") +
  facet_wrap(~ EVAlgorithm,
             labeller = labeller(EVAlgorithm = c(
               `1` = "EV Algorithm",
               `0` = "Preference Algorithm"
             ))) +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  labs(title = "Reasons by treatments",
       x = "Item",
       y = "Rel. frecuency",
       fill = "Never Delegate") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

ggsave("ReasonsByTrmtAndNeverDelegator.png",
       plot = fig46,
       width = 8, height = 4,    # in inches
       dpi = 300)


click_rates_DelegateHyp <- subjects_long %>%
  group_by(SelfHyp10, item) %>%
  summarise(
    rate = mean(clicked, na.rm = TRUE),
    n = n_distinct(ProlificID),
    .groups = "drop"
  )

ggplot(click_rates_DelegateHyp, aes(x = item, y = rate, fill = factor(SelfHyp10))) +
  geom_col(position = "dodge") +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  labs(x = "Item", y = "Klickrate", fill = "SelfHyp10") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




```r
click_rates_both <- subjects_long %>%
  group_by(sumDeleg0, SelfHyp10, item) %>%
  summarise(
    rate = mean(clicked, na.rm = TRUE),
    n = n_distinct(ProlificID),
    .groups = "drop"
  )

ggplot(click_rates_both, aes(x = item, y = rate, fill = factor(SelfHyp10))) +
  geom_col(position = "dodge") +
  facet_wrap(~ sumDeleg0, labeller = labeller(sumDeleg0 = c(`TRUE` = "Never Delegate",
  ↪   `FALSE` = "Delegated"))) +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  labs(x = "Item", y = "Klickrate", fill = "SelfHyp10") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```




```r
# Correct n per condition
n_labels <- subjects_long %>%
  group_by(sumDeleg0, EVAlgorithm) %>%
  summarise(n = n_distinct(ProlificID), .groups = "drop") %>%
  mutate(
    group = case_when(
      sumDeleg0 == TRUE  & EVAlgorithm == 0 ~ "Prefs--Never-delegators",
      sumDeleg0 == TRUE  & EVAlgorithm == 1 ~ "EV--Never-delegators",
      sumDeleg0 == FALSE & EVAlgorithm == 0 ~ "Prefs--Delegators",
      sumDeleg0 == FALSE & EVAlgorithm == 1 ~ "EV--Delegators"
    ),
    group_label = paste0(group, " (n=", n, ")")
  )

# Named vectors for colours and labels
group_colors <- c(
  "Prefs--Delegators"       = "#9dc3e6",
  "Prefs--Never-delegators" = "#1f4e79",
  "EV--Delegators"          = "#f4b183",
  "EV--Never-delegators"    = "#833c00"
)

label_map <- setNames(n_labels$group_label, n_labels$group)

# Main data with group variable as ordered factor
click_rates_both <- subjects_long %>%
  group_by(sumDeleg0, EVAlgorithm, item) %>%
```

```
  summarise(
    rate = mean(clicked, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(
    group = case_when(
      sumDeleg0 == TRUE  & EVAlgorithm == 0 ~ "Prefs--Never-delegators",
      sumDeleg0 == TRUE  & EVAlgorithm == 1 ~ "EV--Never-delegators",
      sumDeleg0 == FALSE & EVAlgorithm == 0 ~ "Prefs--Delegators",
      sumDeleg0 == FALSE & EVAlgorithm == 1 ~ "EV--Delegators"
    ),
    group = factor(group, levels = names(group_colors))  # enforces bar order
  )

fig4_one <- ggplot(click_rates_both, aes(x = item, y = rate, fill = group)) +
  geom_col(position = position_dodge(width = 0.7), width = 0.7) +  # width == dodge width
  ↪  removes gaps
  scale_fill_manual(
    values = group_colors,
    labels = label_map,
    breaks = names(group_colors),
    name = "Condition"
  ) +
  scale_y_continuous(labels = percent_format(accuracy = 1)) +
  labs(x = "Item", y = "Click rate") +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),
    legend.position = "right"
  )

ggsave("ReasonsByTrmtAndNeverDelegator2.png",
       plot = fig4_one,
       width = 8, height = 4,   # in inches
       dpi = 300)
```

if at all, never-delegators are more satisfied with their Part-1 choices.

Reasons for non-delegation

- die Teil-1-Entscheidungen waren voller fehlerhafter Entscheidungen [measurement nach T1: wie zufrieden/sicher? waren Sie sich mit/in ihren Entscheidungen?] Done. Für die einzelnen Entscheidungen: Je sicherer ich in Teil 1 bzgl. der Entscheidung war, desto eher werde ich an den EV-Algorithmus delegieren (?!?), kein Zusammenhang für den PrefAlgorithm.
- ich traue dem Algorithmus nicht zu, meine Präferenzen einzufangen [measurement? Wahrscheinlichkeiten für einen hypothetischen Teil wie beim 1. Exp?] -> ProbOfBadChoices haben keinen Einfluss auf Never-delegation, aber:

EVAlgorithm -0.036028 0.203132 -0.177362 0.85937853
WithinPartDecisionNumber -0.009924 0.005705 -1.739501 0.08327301 . Confidence.y -0.023419 0.013064 -1.792664 0.07432937 . ProbOfAbsurdComputerChoices 0.006830 0.018303 0.373168 0.70936379
ProbOfBadComputerChoices -0.058319 0.016170 -3.606519 0.00037995 * **P105_count 0.004795 0.012610 0.380233 0.70412009**
EVAlgorithm:WithinPartDecisionNumber -0.012264 0.008335 -1.471492 0.14251336

EVAlgorithm:Confidence.y 0.041738 0.016028 2.604073 0.00980698 EVAlgorithm:ProbOfAbsurdComputerChoices -0.070006 0.022913 -3.055368 0.00251080 ** EVAlgorithm:ProbOfBadComputerChoices 0.034684 0.020373 1.702429 0.09001442 .

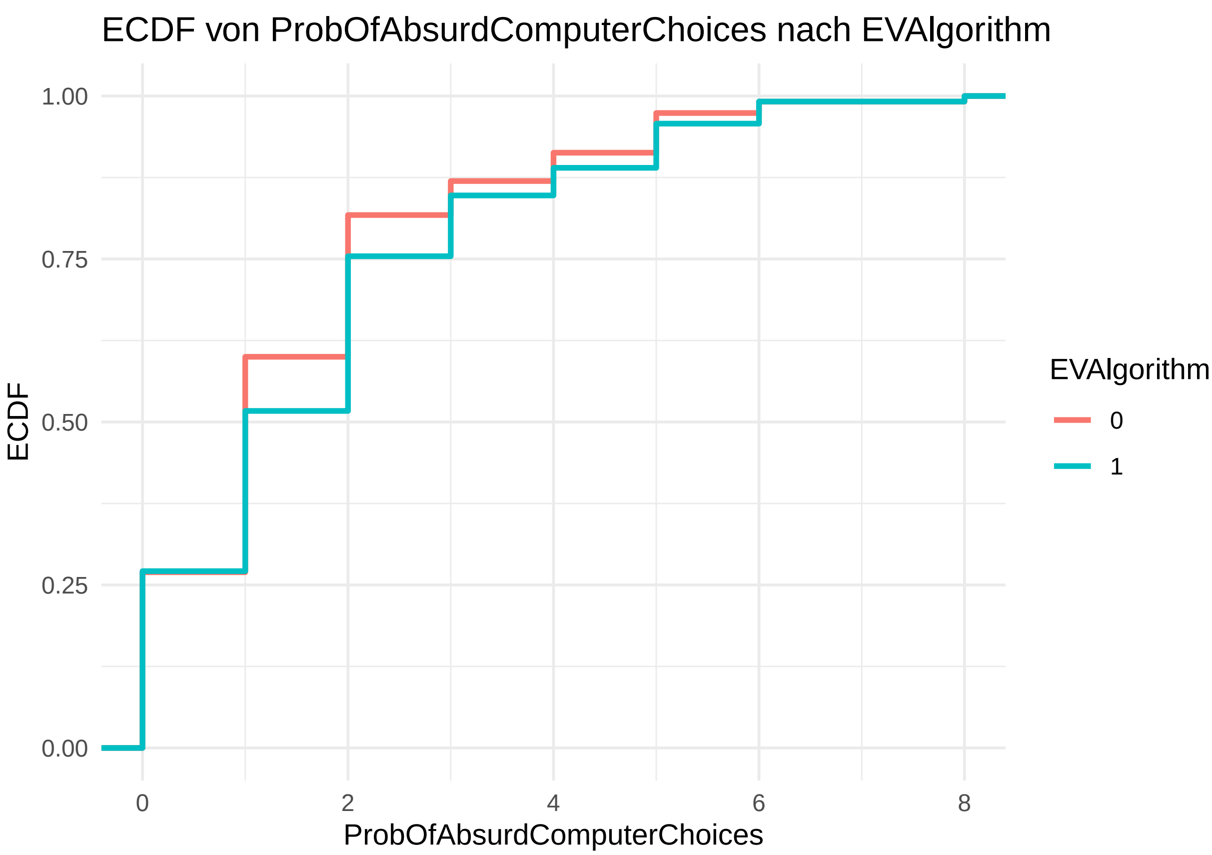
-> im Pref-Alg: je sicherer ich in meiner Lottery-Choice war, desto weniger delegiere ich (p = 0.074) -> im EV-Alg: andersherum! -> ausserdem: Je eher ich dem Computer schlechte Entscheidungen zutraue, desto weniger delegiere ich im Pref-Alg, während im EV-Alg die Angst vor absurden Entscheidungen Delegation verhindert (Das kommt nicht daher, dass im EV-Alg-Trmt keine hohen ProbOfAbsurdComputerChoices vorkommen - das EV-Alg-Trmt first order stochastically dominates the Pref-based treatment in terms of this variable!)

```
table(analysis_data$EVAlgorithm, analysis_data$ProbOfAbsurdComputerChoices)
```

```
##
##      0  1  2  3  4  5  6  8
##   0 31 38 25  6  5  7  2  1
##   1 32 29 28 11  5  8  4  1
```

```
ggplot(analysis_data,
       aes(x = ProbOfAbsurdComputerChoices,
           color = factor(EVAlgorithm))) +
  stat_ecdf(size = 1) +
  labs(
    x = "ProbOfAbsurdComputerChoices",
    y = "ECDF",
    color = "EVAlgorithm",
    title = "ECDF von ProbOfAbsurdComputerChoices nach EVAlgorithm"
  ) +
  theme_minimal()
```

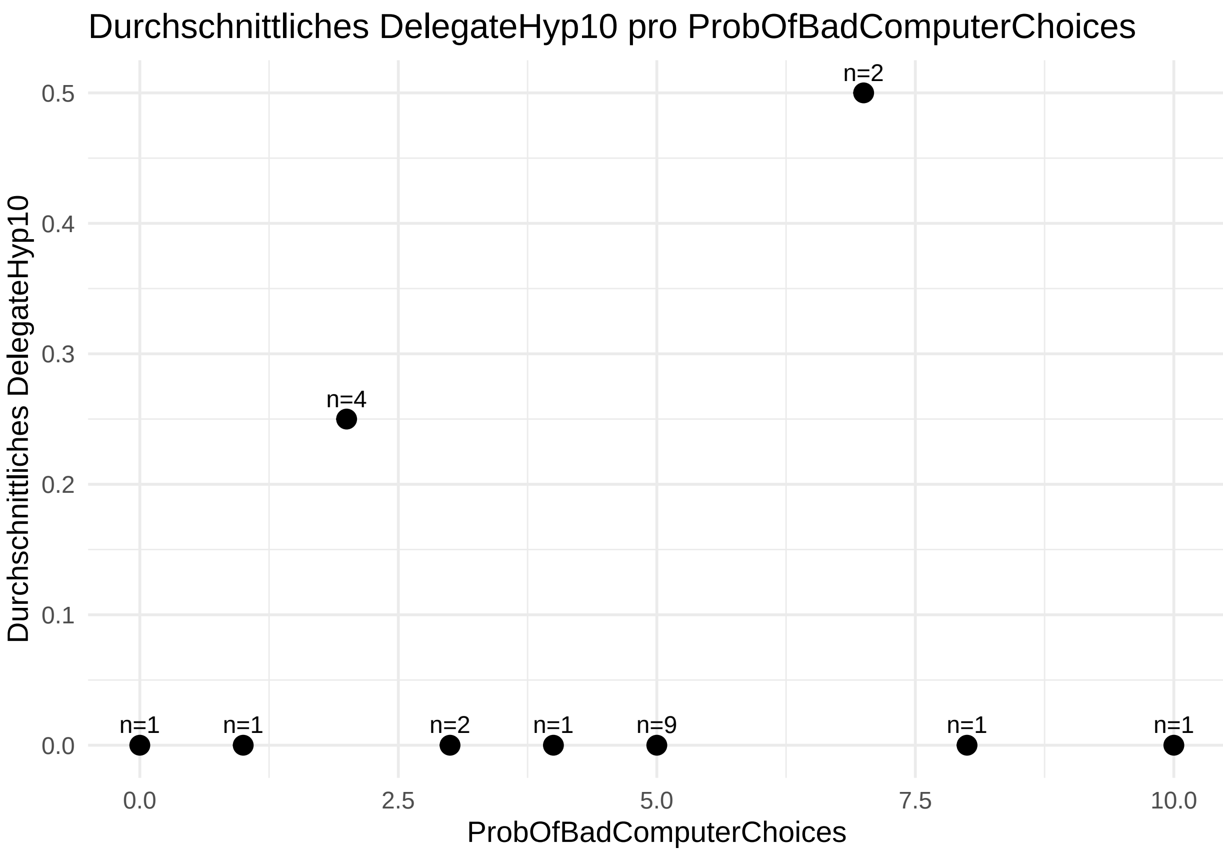
```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



- ich traue dem Algorithmus zu gut zu, meine Präferenzen einzufangen, und das schreckt mich ab [vermutlich das gleiche measurement wie eben] -> eher nein, denn das würde bedeuten, dass bei ProbOfBadComputerChoices == 0 Delegation niedriger wäre als bei höheren Werten, aber das Gegenteil ist der Fall.

```
# 1. ProbOfBadComputerChoices pro Person aus choices
person_data <- choices %>%
  group_by(ProlificID) %>%
  summarise(ProbOfBadComputerChoices = first(ProbOfBadComputerChoices)) %>%
  ungroup()

# Mit DelegateHyp10 und sumDeleg0 joinen
plot_data <- person_data %>%
  left_join(subjects %>% select(ProlificID, DelegateHyp10), by = "ProlificID") %>%
  left_join(NeverDelegate %>% select(ProlificID, sumDeleg0), by = "ProlificID") %>%
  filter(sumDeleg0 == TRUE)    # nur Personen, die sumDeleg0 == TRUE haben

# 3. Mitteln pro ProbOfBadComputerChoices
avg_data <- plot_data %>%
  group_by(ProbOfBadComputerChoices) %>%
  summarise(
    DelegateHyp10_mean = mean(DelegateHyp10, na.rm = TRUE),
    n = n()
  )

ggplot(avg_data, aes(x = ProbOfBadComputerChoices, y = DelegateHyp10_mean)) +
  geom_point(size = 3) +
```

```r
geom_text(aes(label = paste0("n=", n)), vjust = -0.7, size = 3) +
labs(
  title = "Durchschnittliches DelegateHyp10 pro ProbOfBadComputerChoices",
  x = "ProbOfBadComputerChoices",
  y = "Durchschnittliches DelegateHyp10"
) +
theme_minimal()
```



-> für die NeverDelegators kann man es nicht so wirklich angucken (die delegieren ja nicht); Auch in der DelegateHyp10 haben die meisten eine 0, da gibt es keinen klaren Zusammenhang

- ich will nicht von Algorithmen analysiert werden -> wird von keinem NeverDelegator als Grund angeführt, auch unter den anderen so gut wie nie angeklickt.
- ich will nicht, dass Algorithmen mir in meine Entscheidungen 'reinpfuschen -> wird von wenigen angeklickt (unter den NeverDelegators noch weniger)
- Do we have any measure of how complex the lottery-selection task was in people's view? -> anecdotal from the comments: many said time was not enough to make a good decision
- Do we have numbers on average payoffs as a function of AR, by treatment?

```r
library(doBy)
```

```
##
## Attache Paket: 'doBy'

## Das folgende Objekt ist maskiert 'package:dplyr':
##
##     order_by
```

```r
summaryBy (RelevantPayoff ~ Delegated, choices)
```

```
## # A tibble: 2 x 2
##   Delegated RelevantPayoff.mean
##       <int>               <dbl>
## 1         0                195.
## 2         1                243.
```

-> on average, delegation led to a payoff of 243.49 points, non-delegation to a payoff of 195.18 points (25% increase!)

reasons (for hypothetical [non-]delegation) and categorical non-delegation

LPM: never-delegation on reasons

```r
library(lmtest)
```

```
## Lade nötiges Paket: zoo

##
## Attache Paket: 'zoo'

## Die folgenden Objekte sind maskiert von 'package:base':
##
##     as.Date, as.Date.numeric
```

```r
library(sandwich)

reason_vars <- c("DontTrust", "BetterJob", "HowExactly", "MoreInfo",
                 "Responsibility4Choice", "Responsibility4Outcome",
                 "DislikeRelyingOn", "Reactance2Analysis",
                 "PerformanceCuriosity", "Tedious", "Other")

# --- 1. Bivariate LPMs (one per reason, HC2 SEs) ---
bivariate_tbl <- do.call(rbind, lapply(reason_vars, function(rv) {
  m  <- lm(as.formula(paste("as.numeric(sumDeleg0) ~", rv)), data = subjects2)
  ct <- coeftest(m, vcov = vcovHC(m, type = "HC2"))
  data.frame(reason = rv,
             coef   = round(ct[2, 1], 3),
             se     = round(ct[2, 2], 3),
             t      = round(ct[2, 3], 3),
             p      = round(ct[2, 4], 3))
}))
print(bivariate_tbl, row.names = FALSE)
```

```
##                  reason   coef    se      t     p
##               DontTrust  0.068 0.044  1.528 0.128
##               BetterJob -0.131 0.048 -2.739 0.007
##              HowExactly  0.073 0.077  0.950 0.343
##                MoreInfo -0.018 0.049 -0.376 0.707
##   Responsibility4Choice  0.087 0.041  2.111 0.036
##  Responsibility4Outcome -0.068 0.041 -1.688 0.093
##        DislikeRelyingOn -0.070 0.040 -1.769 0.078
```

```
##      Reactance2Analysis -0.097 0.020 -4.926 0.000
##    PerformanceCuriosity -0.013 0.038 -0.328 0.743
##                 Tedious -0.105 0.021 -4.945 0.000
##                   Other -0.001 0.056 -0.014 0.989
```

```
# --- 2. Joint LPM: full sample + preference-treatment only ---
f_joint   <- as.formula(paste("as.numeric(sumDeleg0) ~",
                              paste(reason_vars, collapse = " + ")))
lpm_all   <- lm(f_joint, data = subjects2)
lpm_pref  <- lm(f_joint, data = subjects2[subjects2$EVAlgorithm == 0, ])

vcov_all  <- vcovHC(lpm_all,  type = "HC2")
vcov_pref <- vcovHC(lpm_pref, type = "HC2")

screenreg(
  list(lpm_all, lpm_pref),
  override.se      = list(sqrt(diag(vcov_all)),  sqrt(diag(vcov_pref))),
  override.pvalues = list(
    2 * pt(abs(coef(lpm_all)  / sqrt(diag(vcov_all))),
           df = lpm_all$df.residual,  lower.tail = FALSE),
    2 * pt(abs(coef(lpm_pref) / sqrt(diag(vcov_pref))),
           df = lpm_pref$df.residual, lower.tail = FALSE)
  ),
  custom.model.names = c("Full sample", "Pref. treatment"),
  caption = "LPM: never-delegation ~ reasons (HC2 SEs)"
)
```

```
##
## ====================================================
##                         Full sample  Pref. treatment
## ----------------------------------------------------
## (Intercept)               0.14 **      0.20 *
##                          (0.05)       (0.09)
## DontTrust                 0.08         0.15
##                          (0.04)       (0.09)
## BetterJob                -0.11 *      -0.14
##                          (0.05)       (0.08)
## HowExactly                0.04         0.05
##                          (0.08)       (0.10)
## MoreInfo                 -0.04        -0.15
##                          (0.06)       (0.11)
## Responsibility4Choice     0.06         0.11
##                          (0.04)       (0.07)
## Responsibility4Outcome   -0.04        -0.00
##                          (0.04)       (0.10)
## DislikeRelyingOn         -0.09 *      -0.25 **
##                          (0.04)       (0.08)
## Reactance2Analysis       -0.09        -0.12
##                          (0.04)       (0.12)
## PerformanceCuriosity      0.02         0.03
##                          (0.04)       (0.07)
## Tedious                  -0.07 **     -0.16 **
##                          (0.03)       (0.06)
## Other                    -0.02        -0.01
```

```
##                            (0.06)       (0.12)
## -----------------------------------------------------
## R^2                         0.10         0.16
## Adj. R^2                    0.05         0.06
## Num. obs.                 233          115
## =====================================================
## *** p < 0.001; ** p < 0.01; * p < 0.05
```

```
subjects2$OtherText[subjects2$sumDeleg0 == 1]
```

```
##  [1] ""
##  [2] ""
##  [3] ""
##  [4] ""
##  [5] "It's more fun to choose it myself"
##  [6] ""
##  [7] ""
##  [8] "I am having fun with this and enjoy the risk"
##  [9] ""
## [10] ""
## [11] ""
## [12] ""
## [13] ""
## [14] "I'm just happy to see what happens"
## [15] ""
## [16] ""
## [17] ""
## [18] ""
## [19] ""
## [20] ""
## [21] ""
## [22] ""
```

```
subjects2$Other[subjects2$sumDeleg0 == 1]
```

```
##  [1] 0 0 0 0 1 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0
```

```
subjects2$OtherText[subjects2$sumDeleg0 == 0]
```

```
##   [1] "I analytically quickly analysed and trusted my own judgement, and the more risk averse likely
##   [2] "I felt that with the limited time to make decisions the computer is better suited to calculat
##   [3] ""
##   [4] "not enough time given to fully anaylse data myself"
##   [5] ""
##   [6] ""
##   [7] ""
##   [8] ""
##   [9] ""
##  [10] ""
##  [11] ""
##  [12] ""
##  [13] "I would need more time to view all the options available"
##  [14] "It is just more interesting and gives me something to do in the task. There didn't appear to 
##  [15] "it was a lot of info to absorb in the time available!"
##  [16] ""
```

```
## [17] ""
## [18] "I would have had to make the decision too quickly"
## [19] ""
## [20] ""
## [21] ""
## [22] ""
## [23] ""
## [24] ""
## [25] ""
## [26] ""
## [27] ""
## [28] "I wanted to reduce variance my choosing options with fewest 0 outcomes. The computer's averag
## [29] ""
## [30] ""
## [31] ""
## [32] ""
## [33] ""
## [34] ""
## [35] ""
## [36] "Not enough time to work out the best option myself."
## [37] ""
## [38] ""
## [39] "Not enough time to do it properly myself"
## [40] ""
## [41] "I didn't like the time pressure - it didn't feel like long enough. "
## [42] ""
## [43] ""
## [44] ""
## [45] ""
## [46] ""
## [47] ""
## [48] ""
## [49] ""
## [50] ""
## [51] ""
## [52] ""
## [53] ""
## [54] ""
## [55] ""
## [56] ""
## [57] ""
## [58] ""
## [59] ""
## [60] ""
## [61] ""
## [62] "I would take a higher chance of a low win over a lower chance of a high win, the computer can
## [63] ""
## [64] ""
## [65] "I liked the surprise"
## [66] ""
## [67] ""
## [68] ""
## [69] ""
## [70] "I think the computer could apply better logic than me."
```

```
##  [71] ""
##  [72] ""
##  [73] ""
##  [74] ""
##  [75] ""
##  [76] ""
##  [77] ""
##  [78] ""
##  [79] "The table flashed off to quickly for me to make a proper decision"
##  [80] ""
##  [81] ""
##  [82] ""
##  [83] ""
##  [84] ""
##  [85] ""
##  [86] ""
##  [87] ""
##  [88] ""
##  [89] ""
##  [90] ""
##  [91] ""
##  [92] ""
##  [93] ""
##  [94] "I didn't have enough time to properly assess each of the options myself"
##  [95] ""
##  [96] ""
##  [97] ""
##  [98] ""
##  [99] ""
## [100] ""
## [101] "it was very similar to my choices "
## [102] ""
## [103] ""
## [104] ""
## [105] ""
## [106] "I didn't have enough time too analyse the numbers"
## [107] ""
## [108] ""
## [109] ""
## [110] ""
## [111] ""
## [112] ""
## [113] "Thought the computer would be fun."
## [114] "none"
## [115] ""
## [116] ""
## [117] ""
## [118] ""
## [119] ""
## [120] ""
## [121] ""
## [122] ""
## [123] ""
## [124] ""
```

```
## [125] "there wasn't enough time to choose properly on my own"
## [126] ""
## [127] ""
## [128] ""
## [129] ""
## [130] ""
## [131] ""
## [132] ""
## [133] ""
## [134] ""
## [135] ""
## [136] ""
## [137] "I'm very indecisive and would 'freeze' and run out of time."
## [138] ""
## [139] ""
## [140] ""
## [141] ""
## [142] ""
## [143] ""
## [144] "I felt too anxious with the time limit and found it hard to pick the best option"
## [145] ""
## [146] ""
## [147] ""
## [148] "The time constraints were too strict to digest all of the options fully when selecting myself
## [149] ""
## [150] ""
## [151] ""
## [152] ""
## [153] ""
## [154] ""
## [155] ""
## [156] "There was not enough time to make the decision myself"
## [157] ""
## [158] ""
## [159] "Not given long enough to study the options and make an informed decision myself."
## [160] ""
## [161] ""
## [162] ""
## [163] ""
## [164] "I get satisfaction out of something/somebody else doing something for me/on my behalf. i was
## [165] ""
## [166] "the timing was too short to make a decision in time "
## [167] ""
## [168] ""
## [169] "there wasn't enough time for me to properly weigh up the options"
## [170] ""
## [171] ""
## [172] ""
## [173] ""
## [174] ""
## [175] ""
## [176] ""
## [177] ""
## [178] ""
```

```
## [179] ""
## [180] ""
## [181] ""
## [182] ""
## [183] ""
## [184] ""
## [185] "Due to the time constraints I thought the computer would be able to choose an answer quicker t
## [186] ""
## [187] ""
## [188] ""
## [189] ""
## [190] ""
## [191] ""
## [192] ""
## [193] "i could do the same job in the same time so didn't need the computer to pick it for me but may
## [194] ""
## [195] ""
## [196] ""
## [197] "I cannot scan and choose the best option in the time allotted, so trust that the computer will
## [198] ""
## [199] ""
## [200] ""
## [201] "computer automatically chooses highest total average payoff, so i dont have to calculate it my
## [202] ""
## [203] ""
## [204] ""
## [205] ""
## [206] ""
## [207] ""
## [208] ""
## [209] ""
## [210] ""
## [211] ""
```